

A Project Report On

CYBER ATTACK PREDICTION: FROM TRADITIONAL ML TO GENERATIVE AI

FOR THE PARTIAL FULFILLMENT OF BCA DEGREE UNDER BERHAMPUR
UNIVERSITY

(2025-2026)



**ROLAND INSTITUTE OF COMPUTER & MANAGEMENT
STUDIES**
(SURYA VIHAR, GOLANTHARA, BERHAMPUR, ODISHA, 761008)

Submitted By: -

KUMAR SREYAN PATTANAYAK (Roll: 23PBCA1355)

Guide: Mr. Rasmi Roy Badakumar

GitHub Source Code: [Ksreyan0725/Cyberattack Prediction---College Project: BCA Final Year Project \(2023-2026\) - Cyber Attack Prediction using Random Forest and Generative AI insights.](https://github.com/Ksreyan0725/Cyberattack-Prediction---College-Project-BCA-Final-Year-Project-(2023-2026)-Cyber-Attack-Prediction-using-Random-Forest-and-Generative-AI-insights)

In memory of

Late Dr J Surya Rao "Founder Chairman" RIT Our deepest gratitude.

BONAFIDE CERTIFICATE

This is to certify that the project titled **CYBER ATTACK PREDICTION: FROM TRADITIONAL ML TO GENERATIVE AI** is a Bonafide record of the work done by

KUMAR SREYAN PATTANAYAK (Roll No: 23PBCA1355, Regd. No: 23467/2023)

in partial fulfillment of the requirements for the award of the degree of
Bachelor of Computer Applications (BCA) of the ROLAND
INSTITUTE OF COMPUTER AND MANAGEMENT STUDIES,
SURYA VIHAR, GOLANTHARA, BRAHMAPUR, ODISHA,
761008 during the year 2025-2026.

PRINCIPAL

Guide

Project Viva-voce held on _____

Internal Examiner

External Examiner

Declaration

I hereby declare that, this project entitled “**CYBER ATTACK PREDICTION: FROM TRADITIONAL ML TO GENERATIVE AI**” submitted to Roland Institute of Computer & Management Studies which is affiliated to **BERHAMPUR UNIVERSITY, BHANJA BIHAR, BRAHMAPUR-07 ODISHA, INDIA** in a partial fulfillment for the award of **BACHELOR OF COMPUTER APPLICATIONS (BCA)** degree, is a record of GENUINE WORK done by me under the guidance of **Mr. RASMI ROY BADAKUMAR**, and that the project work has not formed on the basis for the fulfillment of award of any other degree, of fellowship or similarly title to any candidate of any other university.

KUMAR SREYAN PATTANAYAK

ACKNOWLEDGEMENT

IT IS A PRIVILEGE TO WORK UNDER THE GUIDENCE OF (Mr. Rasmi Roy Badakumar) WHO ALWAYS MOTIVATED US TO THINK WITH ADVERSITY AND TO CREATE HUMBLY WITH PERSEVERANCE. HIS EXPECTATIONS, CAPABILITY OF REDUCING COMPLEXITY TO SIMPLER FORM HAS ENOURMOUSLY HELD IN SMOOTH PROGRESS OF THE WORK. IT IS AN ENLIGHTENING EXPERIENCE TO LEARN AND CREATE UNDER HIS CARING ATTITUDE INSTEAD OF BEING BUSY.

*WE WANT TO EXPRESS OUR SINCERE THANKS TO OUR RESPECTED PRINCIPAL, **RABINDRA NATHA NAYAK AND RESPECTED TEACHERS** FOR THEIR CONTINUE DRIVE FOR BETTER QUALITY IN EVERYTHING THAT HAPPENS, AT **ROLAND INSTITUTE OF COMPUTER & MANAGEMANT STUDIES**. THIS REPORT IS A SMALL CONTRIBUTION TOWARDS THE GREATER GOAL OF ACHIEVING EXCELLENCE AT **ROLAND INSTITUTE OF COMPUTER & MANAGEMENT STUDIES**. WE WANT TO EXPRESS OUR THANKS TO THE STAFF OF ROLAND INSTITUTE OF COMPUTER AND MANAGEMENT STUDIES FOR THEIR COOPERATION. WE ENJOYED EVERY MOMENT WORKING WITH THEM.*

WE TAKE THE OPPORTURNITY TO OBLIGE OUR PARENTS FOR THEIR VALUABLE ENCOURAGEMENT DURING STUDY OF BCA IN THIS INSTITUTE. FINALLY, WE WOULD LIKE TO THANK ALL OUR FRIENDS AND THOSE WHO HELPED DIRECTLY OR INDIRECTLY IN PROGRESS AND COMPLETION OF THIS WORK.

Thank You

KUMAR SREYAN PATTANAYAK

TABLE OF CONENTS

1. ABSTRACT	1
2. INTRODUCTION	2
2.1 HOW MACHINE LEARNING WORKS.....	6
2.1.1 Machine Learning Algorithms	7
2.1.2 Types of Machine Learning Methods.....	8
2.1.3 Applications of Machine Learning.....	9
2.1.4 WHAT IS DEEP LEARNING?	11
2.1.4 How Deep Learning Works.....	12
3. LITERATURE SURVEY	13
A] Akasaki et al., 2025	13
1. Introduction to the Theoretical Foundation	13
2. Conceptual Basis of the Theory.....	13
3. Theoretical Constructs Introduced.....	14
4. Methodological Assumptions of the Framework.....	14
B] Sai et al., 2024	15
1. Introduction to Theoretical Context	15
2. Theoretical Basis and Motivation	15
3. Key Theoretical Constructs	16
4. Theoretical Model Architecture	16
5. Theoretical Principles Underlying the Framework	17
C] Arreche et al., 2024	18
1. Introduction to Theoretical Context	18
2. Theoretical Basis and Motivation	18
3. Key Theoretical Constructs	18
4. Theoretical Architecture of E-XAI Framework	19
5. Theoretical Principles.....	20
D] Keshk et al., 2023	20
1. Introduction to Theoretical Context	20
2. Theoretical Motivation and Rationale	21
3. Theoretical Constructs Introduced.....	21
4. Theoretical Architecture of AI-Driven Cybersecurity	22
5. Theoretical Principles and Laws	22
E] Li et al., 2022.....	23

1. Introduction to Theoretical Context	23
2. Theoretical Basis and Motivation	24
3. Key Theoretical Constructs	24
4. Theoretical Architecture of the Deep IDS Model	24
5. Core Theoretical Principles	25
4. SYSTEM ANALYSIS	26
4.1 EXISTING SYSTEM	26
4.2 PROPOSED SYSTEM	26
5. FUNCTIONAL REQUIREMENTS	28
5.1 SYSTEM FUNCTIONAL REQUIREMENTS	28
6 NON-FUNCTIONAL REQUIREMENTS	31
6.1 REQUIREMENTS	31
7. FEASIBILITY STUDY	33
8. REQUIREMENT SPECIFICATION	34
8.1 Hardware Requirements	34
8.2 Software Requirements	34
9 SYSTEM DESIGN	35
9.1 METHODOLOGY	35
9.1 DATASET	38
9.3 SYSTEM ARCHITECTURE	40
10 UML DIAGRAMS	41
10.1 USE CASE DIAGRAM	42
10.2 CLASS DIAGRAM	43
10.3 SEQUENCE DIAGRAM	44
10.4 COLLABORATION DIAGRAM	45
10.5 ACTIVITY DIAGRAM	46
11 TECHNOLOGY DESCRIPTIONS	47
11.1 PYTHON HISTORY	47
11.2 PYTHON FEATURES	48
11.3 PYTHON APPLICATIONS	50
11.4 WHY PYTHON	51
11.5 PYTHON INSTALLATION	52
12. SAMPLE CODE	57
13. TESTING	69
13.1 INTRODUCTION	69
13.1.1 System Testing and Implementation	69
13.1.2 Testing Techniques	70

13.1.3 Testing Strategies	71
14. SCREENSHOTS	73
15. CONCLUSION	82
FUTURE SCOPE.....	82
16. BIBLIOGRAPHY.....	83

LIST OF FIGURES

Figure 1 Architecture	40
Figure 2 Use case Diagram	42
Figure 3 Class Diagram	43
Figure 4 Sequence Diagram.....	44
Figure 5 Collaboration Diagram.....	45
Figure 6 Activity Diagram	46

LIST OF SCREENS

Screen 1 Importing required python classes and packages.....	73
Screen 2 Displaying CICIDS2017 dataset values.....	73
Screen 3 Displaying CICDDOS2019 dataset values.....	74
Screen 4 Displaying XIOTIOT dataset values.....	74
Screen 5 Visualizing graph of different attacks found in dataset.....	75
Screen 6 Training Random Forest.....	75
Screen 7 KNN.....	76
Screen 8 SVM.....	76
Screen 9 MPL	77
Screen 10 Logistic Regression.....	77
Screen 11 Naïve Bayes	78
Screen 12 DNN.....	78
Screen 13 Training extension hybrid algorithms	79
Screen 14 SHAP explanation tool.....	79
Screen 15 Showing all algorithms accuracy	80
Screen 16 Showing comparison graph between all algorithms	80
Screen 17 Attack prediction	81

1. ABSTRACT

The extended study on *Cyber Attack Prediction* enhances the base model by implementing an advanced hybrid approach that integrates multiple machine learning algorithms, including MLP, KNN, and Random Forest, using a stacked classifier technique. This extension aims to achieve higher accuracy and robustness in detecting complex cyber threats across multiple benchmark datasets such as NSL-KDD, CICIDS2017, CICDDOS2019, and XIIOT. The hybrid model successfully improved prediction performance, achieving an impressive accuracy rate of 99–100%, outperforming individual traditional algorithms. By combining the strengths of various models, the system effectively identifies a wide range of attack patterns with improved precision and reduced false alarms. Additionally, SHAP explainability tools were used to interpret the model's decisions, ensuring transparency and trust in predictions. This extended framework provides a more reliable, scalable, and intelligent solution for real-time cyber-attack detection in evolving digital environments.

2. INTRODUCTION

In the digital era, where information systems form the backbone of almost every sector, the threat of cyber-attacks has emerged as one of the most significant challenges faced by governments, organizations, and individuals. The increasing dependence on technology has enhanced productivity, efficiency, and connectivity, but it has also made networks and systems more vulnerable to sophisticated cyber threats. With the exponential growth in the volume of data and the proliferation of interconnected devices, cybercriminals have found new and advanced methods to exploit vulnerabilities and breach security frameworks. Consequently, cybersecurity has become an essential pillar in the modern world, aimed at safeguarding sensitive information, maintaining system integrity, and ensuring uninterrupted service availability.

Cyber-attacks are deliberate, malicious attempts to compromise the confidentiality, integrity, or availability of information systems. These attacks can take numerous forms—ranging from ransomware, phishing, and denial-of-service attacks to advanced persistent threats, malware, and zero-day vulnerabilities. In recent years, the frequency, intensity, and sophistication of these attacks have increased dramatically. Events such as the Colonial Pipeline ransomware attack in 2021, the SolarWinds supply-chain breach in 2020, and the WannaCry ransomware outbreak in 2017 have demonstrated the devastating financial, social, and geopolitical impact of cybercrimes. These incidents disrupted critical services, caused massive financial losses, and exposed sensitive information, illustrating the global implications of weak cybersecurity mechanisms.

Traditional cybersecurity methods primarily rely on predefined signatures, heuristic rules, and reactive mechanisms to detect and respond to attacks. While these techniques have been effective in identifying known threats, they often fail to adapt to the rapidly evolving landscape of cybercrime, where attackers continuously innovate new strategies and exploit zero-day vulnerabilities. As cyber-attacks become more complex and less predictable, the demand for intelligent, adaptive, and proactive defense mechanisms has become imperative. This shift has paved the way for the integration of Artificial Intelligence (AI) into cybersecurity frameworks.

Artificial Intelligence (AI) has revolutionized the field of cybersecurity by introducing systems that can learn from data, identify patterns, and make informed decisions without

explicit programming. AI encompasses several subfields—Machine Learning (ML), Deep Learning (DL), Natural Language Processing (NLP), and Explainable AI (XAI)—each contributing unique capabilities to cyber-attack detection and prevention. Machine Learning algorithms, in particular, have demonstrated remarkable success in automating the detection of anomalies, classifying attack types, and predicting potential threats based on historical data. These algorithms can process vast amounts of structured and unstructured data, recognize subtle correlations, and make accurate predictions that traditional methods might overlook.

Machine Learning (ML) techniques such as Support Vector Machines (SVM), Random Forests (RF), K-Nearest Neighbors (KNN), Logistic Regression (LR), and Naïve Bayes (NB) are extensively used for attack classification, intrusion detection, and anomaly detection. By learning from labeled datasets like NSL-KDD, CICIDS2017, and CICDDoS2019, these models can identify patterns associated with normal and malicious network behavior. Deep Learning (DL), a specialized subset of ML, further enhances predictive performance by using multi-layered neural networks capable of learning complex feature representations. Models like Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Deep Neural Networks (DNN) are used for real-time intrusion detection, malware analysis, and phishing detection with exceptional accuracy. Unlike traditional systems, DL models can automatically extract hierarchical features from raw data, significantly improving detection rates for unseen attack patterns.

However, the application of AI in cybersecurity is not without challenges. One of the primary concerns is the “black-box” nature of many AI and DL models, which limits transparency and interpretability. In cybersecurity, where decisions directly impact the safety of data and infrastructure, understanding the rationale behind AI predictions is vital. This challenge has led to the emergence of Explainable AI (XAI), an approach designed to make AI decisions interpretable and trustworthy. XAI provides clear explanations for predictions, identifies influential features, and enhances user confidence in AI-driven cybersecurity systems. Tools like SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) are increasingly being used to analyze which features contribute most to attack detection, ensuring accountability and reliability in automated decision-making systems.

The evolution of cyber threats also coincides with the rise of Generative Artificial Intelligence (GenAI), which adds a new dimension to both cybersecurity offense and defense.

Generative AI models, powered by advanced neural architectures such as Generative Adversarial Networks (GANs) and Transformers, can create realistic synthetic data, simulate attack scenarios, and generate threat intelligence. While GenAI can assist in developing more resilient defensive mechanisms, it also introduces new risks, as malicious actors can exploit these tools to generate sophisticated phishing messages, deepfake videos, or adaptive malware. Therefore, understanding and managing the dual-use nature of GenAI is a crucial aspect of modern cybersecurity research.

In response to these challenges, researchers have been exploring hybrid and ensemble learning models that combine the strengths of multiple algorithms. These models, such as stacked classifiers that integrate base learners like KNN, MLP, and Random Forest, offer superior accuracy and robustness in detecting cyber-attacks. By leveraging the diversity of models and datasets, hybrid approaches can effectively handle variations in attack types and improve overall prediction reliability. The combination of ML, DL, and XAI techniques forms the foundation for a next-generation cybersecurity framework that is proactive, interpretable, and resilient.

The availability of high-quality benchmark datasets has played a crucial role in advancing cyber-attack prediction research. Datasets such as NSL-KDD, KDDCUP99, CICIDS2017, CICDDoS2019, and IoT-based traffic datasets enable researchers to train and evaluate models across diverse attack scenarios. These datasets encompass various types of network traffic and attack behaviors, providing a comprehensive foundation for experimentation. However, issues such as data imbalance, outdated attack patterns, and limited real-world diversity still persist, affecting the generalizability of AI models. Consequently, continuous dataset enrichment and the integration of real-time data streams are essential for maintaining the relevance and effectiveness of AI-driven cybersecurity systems.

Another critical dimension of cyber-attack prediction is the real-time analysis and response capability. AI-driven systems can monitor massive streams of network traffic, detect anomalies in milliseconds, and initiate automated countermeasures before significant damage occurs. This proactive detection capability marks a shift from reactive defense mechanisms to predictive security frameworks. With the integration of NLP techniques, AI systems can also analyze threat intelligence reports, emails, and communications to detect phishing campaigns and social engineering attempts. This multidisciplinary approach enhances situational awareness and reduces response time in combating cyber incidents.

Despite these advancements, the deployment of AI in cybersecurity raises concerns about data privacy, resource consumption, and adversarial attacks. AI models can themselves become targets of manipulation through data poisoning or evasion attacks, leading to misclassification of malicious data as benign. Therefore, strengthening the robustness and explainability of AI models is essential to ensure trustworthy cybersecurity systems. Moreover, ethical considerations and regulatory frameworks must guide the responsible use of AI to prevent misuse by threat actors.

The role of AI in cybersecurity extends beyond mere automation; it represents a paradigm shift toward intelligent and adaptive security ecosystems. By integrating machine intelligence with human expertise, cybersecurity systems can evolve continuously, learning from new threats and adapting to dynamic environments. Explainable and interpretable models foster trust, while generative and hybrid frameworks enable resilience and innovation in defense mechanisms. As cyber threats become more intricate and borderless, the fusion of AI technologies offers an unprecedented opportunity to predict, prevent, and neutralize attacks effectively.

2.1 HOW MACHINE LEARNING WORKS

To begin with, this and computer science and, more particularly, AI and its subset called machine learning focuses on the collection of data and making use of algorithms to accomplish the objective of replicating the human learning processes more accurately rather than attempting to entirely imitate them. This is critical for data science as it allows systems to be able to uncover complex relationships within data that can make predictions or classifications automatically, and later using these relationships to make decisions within a business or any other establishment. Such insights are essential, especially for data mining initiatives and can have critical impact on major business development indices.

However, as the volume of big data is increasing so does the need for data scientists who understand the business perspective and the problems that needs solving and how data can provide the answers to the problems. This is where machine learning becomes relevant as it provides an opportunity to solve all these problems by automating the process through the use of statistical methods in training algorithms to recognize underlying relationships or patterns with data.

2.1.1 Machine Learning Algorithms

There are three important principles on which machine learning algorithms evolve.

1. A Decision Process.

This is the formulization of the problem which has to be solved by the machine learning. The problem solving begins with the identification of what the problem is and this means stating what are the objectives of the problem

2. An Error Function

The algorithm's predictions are ascertained with the aid of the error function. The error function is an evaluation tool, which verifies the efficiency of the model based on the output in relation to the available known situations and indicates places where improvements can be made in the model.

3. A Model Optimization Process

During the optimization process, the already defined model is employed to enhance its accuracy by modifying parameters, for instance, weights, that determine the alignment of the model with the data. The model conducts its own evaluation, causes some changes which are afterward subjected to further evaluation, and repeats everything until it fully satisfies itself with the accuracy of its prediction, or the process exceeds some limits preset.

2.1.2 Types of Machine Learning Methods

Supervised Machine Learning

Initiating with labelled data leads to supervised learning algorithms, thereby informing the input and the output. In this manner, the model is able to understand the rules and the relationships of the data so as to classify patterns and provide predictions of unseen data. Adding the data would energise the model to make weight adjustments to the parameters so as to better the performance during the training phase. Cross-validation is adopted for balancing cases where overfitting (not too general) and underfitting situations (too general) are avoided.

Unsupervised Machine Learning

On the contrary, unsupervised learning involves working with unlabelled sets of data. Such algorithms examine the input data to discover and cluster the data into groups or structures that were not taught to them. This makes it very useful for exploratory data analysis, for market segmentation, cross-marketing, or for recognizing images and patterns.

Semi-Supervised Learning

To do this, semi-supervised learning bridges supervised and unsupervised learning by employing a few labeled data and lots of unlabeled data. The final understanding of the academic efforts is achieved through feature extraction and image collection from the labeled files, hence the model is able to understand the unpreferred files. This model proves useful in situations where the labeled data is scarce or the processes of labeling data is expensive or infeasible.

2.1.3 Applications of Machine Learning

1. Voice Identification

Automatic Speech Recognition (ASR) also known as voice recognition is that aspect of natural language processing (NLP) wherein spoken words are converted to written texts. This technology is quite widely used on portable devices, for instance, mobile phones, where people are able to do voice searches on devices such as Siri and can also send text messages without using their hands.

2. Customer support

Customer interaction and engagement are being revolutionized by AI powered chatbots. Customers can be assisted on sites and across social media platforms. These AI powered bots can answer frequently asked questions about shipping conditions, give fitting and size suggestions, and recommend goods to buy, all of which help to maximize customer interaction. These can include ecommerce virtual agents, slack and facebook messenger bots, and voice interface systems.

3. Vision AI

With the help of computer vision, systems can analyze faces, videos and other visual aspects to gain an understanding of the situation at hand. Unlike simple image recognition, such approaches can view pictures and videos and consider them for decision making. These include:

- Tagging of pictures posted on social media sites.
- Healthcare radiology imaging.
- Automobiles with automated driving features in the automobile sector.

4. Virtual shopping assistants

Past behavioral data of users is outputted by an AI algorithm and then analyzed to find valuable insights which can be used to strategically target users. This is most common in e-commerce stores where clients are offered recommendations about items that can be purchased in addition to what the user initially intended to buy.

5. Stock trading robots

The new trading platforms that have been developed focus on algorithmic trading of stocks while an AI powered engine assembles the stock portfolio. Daily, these computers can place hundreds of thousands or millions of trades, making them suitable for stock markets.

2.1.4 WHAT IS DEEP LEARNING?

Deep learning is a subfield of AI that engages people's attention the most today. This is the case because deep learning allows the automation of tasks such as image classification, speech recognition, object detection, and content description. In short, deep learning allows machines to comprehend data in a more advanced way.

For example, deep learning is used in image classification, speech recognition, object detection, and content analysis. The exponential growth can be attributed to several factors:

Key Drivers of Deep Learning Advancements

1. Algorithmic Improvements

The innovations in the algorithms have drastically increased the usability of the deep learning models.

2. New Machine Learning Approaches

The methodologies have evolved over the years, and that has retained the prediction accuracy of the models as well as the decisions made.

3. Specialized Neural Networks

New formats of the neural networks have been created to enable the performance of certain functions such as document translation or photos categorization to these networks.

4. Abundance of Metadata

The availability of such data is practically a revolution and includes the following types:

- Streaming data from IoT devices;
- Textual data from social media;
- Professional notes like physicians' notes and investigative transcriptions.

These data sources constitute a solid basis for constructing and deep-layered neural networks.

2.1.4 How Deep Learning Works

The deep learning paradigm has the ability to bring revolution in advancement of the field analytics. Deep learning treats the problem differently, as in this approach instead of being explicitly coded the whole program detailing how the problem can be solved, the clear problem, in this case, is to teach the computer to learn how to solve it by itself.

Analytical Methodology In Deep Learning

In traditional analytics, the steps involved in the approach tend to include the following:

1. **Feature Engineering:** Taking out an variable (features) or picking out variables from the data set and recreating ones that can adequately describe the issue.
2. **Model Development:** All the phases of the study until a particular model is completed and picked on.
3. **Parameter Fitting:** Creating the best model that represents the data by changing the parameters of the model.

Although this approach can yield satisfactory predictive systems, it has weaknesses. The performance of the system is highly influenced by the quality of the features and the model. For example, while constructing a model for detecting fraud, one would say:

- You start with parameters that are already set and apply data manipulations in order to generate a model.
- It follows that this process may produce several thousands of variables, which will involve painstaking work to sift through this mass and select the significant ones.
- Inserting new information requires the application of the entire feature engineering exercise all over again.

3. LITERATURE SURVEY

A) Ankalaki et al., 2025

1. Introduction to the Theoretical Foundation

- The work by **Ankalaki et al. (2025)** presents a transformative theoretical foundation for the evolution of **cyber-attack prediction** through artificial intelligence.
- It unites **traditional machine learning (ML)** methods with **deep learning (DL)**, **Explainable AI (XAI)**, and **Generative AI (GenAI)** into a single conceptual model.
- The theory aims to address the limitations of earlier rule-based and statistical models that failed to detect advanced and zero-day cyber threats.
- The authors propose a **layered intelligence framework** where learning models evolve beyond pattern recognition — toward reasoning, transparency, and autonomous cyber defense.
- Their theoretical contribution lies in redefining cybersecurity from being **reactive** to **predictive and proactive**, leveraging AI to understand, anticipate, and mitigate threats in real time.

2. Conceptual Basis of the Theory

- The theory assumes that **cyber threats are dynamic phenomena** — continually evolving, adapting, and learning — thus requiring adaptive defense mechanisms that can evolve similarly.
- Traditional machine learning techniques (e.g., Decision Trees, Random Forest, and SVM) rely on fixed statistical models, which limit generalization to unseen attacks.
- Ankalaki et al. theorize that by integrating **multi-level AI techniques**, cybersecurity systems can mimic **human cognitive processes**: observing, learning, interpreting, and generating defense strategies.
- The theoretical model includes four AI pillars:
 1. **Machine Learning (ML)**: Pattern-based recognition and anomaly detection.
 2. **Deep Learning (DL)**: Hierarchical feature extraction for complex attack identification.
 3. **Explainable AI (XAI)**: Transparency and interpretability for decision accountability.

- 4. **Generative AI (GenAI):** Simulation and generation of new attack patterns to improve preparedness.
- Together, these layers form an **intelligent, adaptive cyber prediction ecosystem**.

3. Theoretical Constructs Introduced

- **AI-Centric Cyber Defense Paradigm:**
 - The paper introduces an AI-centric defense model in which predictive intelligence replaces signature-based mechanisms.
 - It conceptualizes cybersecurity as a **self-learning feedback system**, where algorithms continuously refine their understanding of new threats.
- **Hybrid Predictive Framework:**
 - The authors propose combining supervised, unsupervised, and generative techniques for comprehensive threat coverage.
 - The theoretical hybridization merges data-driven reasoning (ML/DL) with synthetic data generation (GenAI).
- **Explainability Construct (XAI):**
 - An essential theoretical addition is the incorporation of **explainability** as a core feature of defense models.
 - XAI bridges the trust gap between automated predictions and human analysts by explaining how the model identifies and classifies cyber-attacks.
- **Generative Adversarial Learning:**
 - The GenAI component introduces adversarial modeling, where **AI simulates attacks** to test and improve defenses.
 - This concept extends the “**Red Team vs. Blue Team**” paradigm into the digital domain, creating a perpetual learning loop.

4. Methodological Assumptions of the Framework

- The theoretical structure relies on several key assumptions:
 1. **Data Availability:** Sufficient labeled and unlabeled datasets (e.g., NSL-KDD, CICIDS2017, CICDDoS2019, IoT datasets).
 2. **Dynamic Feature Representation:** Attack characteristics evolve, requiring continuous feature learning.

3. **Interpretable AI Necessity:** Decision transparency must accompany automation for practical deployment.
 4. **Synthetic Data Utility:** Generative models can supplement insufficient real-world data to improve learning diversity.
- These assumptions align with modern cyber-attack prediction models that depend on **massive, evolving datasets and adaptive algorithms.**

B| Sai et al., 2024

1. Introduction to Theoretical Context

- **Sai et al. (2024)** offer a forward-looking theoretical foundation that redefines how **Generative Artificial Intelligence (GenAI)** can transform the landscape of **cybersecurity defense and intelligence.**
- Their central thesis argues that **large language models (LLMs)** and **generative frameworks** (like ChatGPT, DALL·E, and Codex) can both *enhance* and *threaten* cybersecurity ecosystems.
- The theoretical work highlights a **duality**: while GenAI can simulate attacks to strengthen defenses, it can also be exploited by malicious entities to automate cybercrime.
- This framework establishes a **balanced theoretical lens** to understand GenAI as both a tool for prediction and as an object of ethical and technical control.
- The study positions Generative AI as the next evolutionary phase in cybersecurity — moving from reactive detection to **adaptive simulation and proactive prediction.**

2. Theoretical Basis and Motivation

- The paper is grounded in the **AI lifecycle theory** — proposing that every AI capability (generation, perception, reasoning, and adaptation) contributes to cyber defense.
- **Motivation:** The exponential growth of data and the creativity of attackers demand **intelligent systems that can think, predict, and create.**
- **Theoretical Objective:** To integrate generative modeling with cybersecurity defense frameworks, allowing systems to:

1. **Predict potential vulnerabilities.**
 2. **Simulate real-world attacks** for preemptive training.
 3. **Generate realistic threat intelligence datasets.**
- The authors propose that Generative AI's ability to produce contextually coherent outputs provides an unprecedented mechanism for *synthetic training, adversarial testing, and automated defense*.

3. Key Theoretical Constructs

1. Generative-Defensive Paradigm:

- The authors introduce the concept of using **generative models as defensive weapons**.
- These models synthesize data to anticipate adversarial behavior, creating *digital twins* of attack environments.

2. Dual-Use Theory:

- GenAI can serve both defenders and attackers; understanding this duality forms the paper's theoretical crux.
- This notion parallels **game theory**, where both sides continuously evolve strategies through machine-generated intelligence.

3. LLM-based Cyber Intelligence:

- The paper theorizes that LLMs like ChatGPT can extract, interpret, and summarize massive cybersecurity datasets.
- Such systems enable **contextual understanding of threats**, unlike traditional rule-based systems.

4. Adaptive Security Feedback Loop:

- GenAI models learn continuously, forming a **self-improving defense cycle** — where outputs feed back into the learning process to enhance security predictions.

4. Theoretical Model Architecture

Sai et al. propose a **four-layered theoretical model** for integrating Generative AI with cybersecurity defense:

1. **Data Representation Layer:**

- Converts structured and unstructured cybersecurity data (logs, text, images) into model-readable form.
- LLMs process this data using **semantic embeddings**, linking textual and behavioral data.

2. **Generative Simulation Layer:**

- Models like GANs (Generative Adversarial Networks) simulate **malware, phishing, and ransomware** patterns.
- This allows for training detection algorithms on synthetic but realistic data.

3. **Explainability and Control Layer:**

- Introduces **Explainable AI (XAI)** components to interpret GenAI decisions.
- Ensures transparency, mitigating the “black box” issue inherent in deep generative systems.

4. **Feedback Adaptation Layer:**

- Uses reinforcement learning to refine both defensive and generative capabilities dynamically.
- This continuous learning process aligns GenAI with real-time threat evolution.

5. **Theoretical Principles Underlying the Framework**

- **Generativity as Intelligence:** True cyber intelligence requires generative creativity — the ability to hypothesize unseen threats.
- **Symbiotic AI Evolution:** Defense systems must evolve alongside attack models, creating an **AI-vs-AI arms race**.
- **Transparency Imperative:** As AI becomes autonomous, explainability is not optional but ethical necessity.
- **Synthetic Data Validity:** Generative models can ethically substitute real attack data while maintaining learning integrity.
- **Continuous Adaptation:** The security ecosystem must self-evolve, guided by real-time feedback loops.

These principles lay the foundation for **autonomous, adaptive, and explainable AI-driven security systems**.

C| Arreche et al., 2024

1. Introduction to Theoretical Context

- **Arreche et al. (2024)** advance the theoretical discussion of **Explainable Artificial Intelligence (XAI)** in the field of **network intrusion detection (NID)** by introducing a comprehensive evaluation framework known as **E-XAI**.
- Their theory tackles one of the most persistent challenges in cybersecurity — the “black-box” nature of deep learning and ensemble models.
- The core theoretical premise is that **interpretability and transparency** are essential for trust, accountability, and reliability in AI-driven intrusion detection systems (IDS).
- While prior works focused primarily on accuracy improvement, this research introduces a new theoretical dimension: **explainability as a measurable quality of intelligence**.
- The paper lays the groundwork for a structured theoretical model that quantifies and evaluates the interpretability of AI algorithms in cybersecurity.

2. Theoretical Basis and Motivation

- The authors begin from the **theory of cognitive interpretability**, which posits that any intelligent system must produce outputs that are understandable to humans.
- **Motivation:** Despite the high accuracy of modern deep learning models (CNNs, LSTMs, Transformers), they fail to justify their reasoning — making them unsuitable for critical defense domains.
- **Theoretical Goal:** Develop a measurable framework that allows security analysts to evaluate how well an AI system explains its decisions.
- The theory builds upon **trust-based AI governance** and **ethical accountability**, emphasizing that interpretability is as crucial as predictive power.
- The work recognizes explainability not just as a supplementary feature but as a **fundamental theoretical requirement** in cyber defense systems.

3. Key Theoretical Constructs

1. Explainability Spectrum:

- The paper defines a continuum from **transparent models (Decision Trees)** to **opaque models (Deep Neural Networks)**.
- The position of any model on this spectrum determines its level of interpretability.

2. E-XAI Metric Framework:

- Introduces a structured evaluation metric based on **fidelity, clarity, completeness, and consistency**.
- These dimensions collectively form the theoretical core of explainability assessment.

3. Human-Centric AI Theory:

- Proposes that the ultimate goal of cybersecurity AI is **human-aligned decision intelligence**.
- Human interpretability becomes the validation mechanism of model trustworthiness.

4. Black-Box Decomposition Model:

- Suggests decomposing opaque models into interpretable sub-components using tools such as SHAP, LIME, and Grad-CAM.
- The decomposition theory allows reconstructing decision pathways post hoc.

4. Theoretical Architecture of E-XAI Framework

Arreche et al. propose a **four-phase architecture** that functions as both an evaluation and theoretical model:

1. Input Representation Phase:

- Security features (network flows, payloads) are transformed into numerical embeddings.
- The theoretical concern is how feature encoding impacts interpretability.

2. Model Decision Phase:

- AI models generate predictions of attack types or benign traffic.
- The “black-box” layer operates here, where opacity must be theoretically quantified.

3. Explanation Generation Phase:

- Utilizes local and global explanation techniques to map model reasoning to input variables.

- This stage embodies the “**interpretive intelligence**” theory, where AI mimics human reasoning transparency.

4. Evaluation and Scoring Phase:

- Assigns quantitative scores to explainability criteria, forming a benchmark for model comparison.

This four-layer architecture positions E-XAI as a **meta-theoretical framework** — a model that evaluates other models.

5. Theoretical Principles

- **Principle of Transparency:** An AI model is only trustworthy when its decisions can be understood and verified.
- **Principle of Traceability:** Every AI decision must be traceable back to its contributing features.
- **Principle of Balance:** Explainability must not compromise predictive accuracy.
- **Principle of Human Comprehension:** AI explanations should be intuitively understandable, not mathematically abstruse.
- **Principle of Ethical Alignment:** Transparency and accountability are ethical imperatives in AI-based defense systems.

These principles create a **philosophical backbone** for developing explainable and responsible AI in cybersecurity.

D| Keshk et al., 2023

1. Introduction to Theoretical Context

- **Keshk et al. (2023)** present an extensive theoretical framework that unifies **machine learning (ML)**, **deep learning (DL)**, and **hybrid intelligence** for **cybersecurity defense and prediction systems**.
- The core theoretical claim is that cybersecurity should be redefined as a **knowledge-driven adaptive system**, capable of learning continuously and autonomously.
- Their work establishes a **multi-layered theoretical foundation** connecting traditional computational security models with modern AI paradigms.

- The framework advances a conceptual shift from *rule-based static systems* to *intelligent, self-evolving systems* that mimic human cognitive capabilities.
- Keshk et al. emphasize that true cyber defense must integrate **AI reasoning, decision intelligence, and feedback learning**, forming the intellectual groundwork for AI-augmented security.

2. Theoretical Motivation and Rationale

- The motivation arises from the **explosion of complex cyber threats** such as ransomware, advanced persistent threats (APTs), and polymorphic malware that evade static defenses.
- The authors argue that traditional systems — reliant on manually curated signatures — cannot match the **scale, speed, and diversity** of modern attack patterns.
- **Theoretical objective:** to establish a universal model of **AI-Driven Cybersecurity**, where machine learning not only identifies anomalies but also **predicts attack trajectories**.
- The theory positions AI as a **cognitive engine** that learns from historical and real-time data, offering continuous situational awareness.

3. Theoretical Constructs Introduced

1. AI-Centric Security Ecosystem:

- Cybersecurity is theorized as an ecosystem where multiple AI agents interact, learn, and adapt to defend networks collectively.

2. Hybrid Intelligence Model:

- Integrates ML, DL, and expert systems to combine computational accuracy with domain reasoning.

3. Feedback-Driven Adaptation:

- Security models must include feedback loops for self-improvement based on detected anomalies and outcomes.

4. Explainability in Hybrid Systems:

- Transparency is embedded theoretically within hybrid intelligence models, ensuring trustworthiness and interpretability.

5. Adaptive Defense Principle:

- Defense systems must autonomously adjust decision boundaries as threat landscapes evolve.

These constructs form a **cyber intelligence theory** that combines automation, cognition, and self-learning.

4. Theoretical Architecture of AI-Driven Cybersecurity

Keshk et al. propose a **five-layer theoretical model** that outlines how AI integrates with cybersecurity processes:

1. Data Acquisition Layer:

- Collects structured (logs, network packets) and unstructured (emails, texts) data.
- Theoretical emphasis on data diversity as the backbone of intelligent defense.

2. Feature Engineering Layer:

- Transforms raw data into meaningful features using correlation, statistical, and semantic extraction.
- Establishes the theoretical foundation for representation learning in cyber contexts.

3. Learning Layer:

- Employs ML (Random Forest, SVM) and DL (CNN, RNN, LSTM) models to detect known and unknown threats.

4. Decision Optimization Layer:

- Applies reinforcement learning and hybrid reasoning to refine prediction confidence.

5. Knowledge Fusion Layer:

- Integrates insights from various AI subsystems to generate holistic situational intelligence.

This architecture reflects the theoretical evolution from isolated ML models to **collaborative AI ecosystems**.

5. Theoretical Principles and Laws

- **Principle of Continuous Learning:**

- AI systems must evolve autonomously as data and threats change.
- **Principle of Hybrid Cognition:**
 - Combining symbolic reasoning (logic) with sub-symbolic reasoning (neural learning) yields superior intelligence.
- **Principle of Transparency:**
 - Theoretical integrity demands that AI's reasoning process be explainable to human operators.
- **Principle of Human-AI Synergy:**
 - True cybersecurity intelligence arises when human analysts and AI systems co-learn.
- **Principle of Predictive Awareness:**
 - Security systems must anticipate future threats, not merely react to existing ones.

These principles form the **philosophical and logical spine** of Keshk et al.'s theory.

E] Li et al., 2022

1. Introduction to Theoretical Context

- **Li et al. (2022)** establish a foundational theory for applying **deep learning (DL)** in **intrusion detection and cyber-attack prediction**.
- Their work conceptualizes cybersecurity as a **data-driven cognitive process**, where DL architectures emulate the human brain's layered learning ability.
- The authors identify the theoretical gap between traditional ML models—which rely on handcrafted features—and DL models capable of automatic hierarchical feature extraction.
- The central theoretical claim is that **multi-layer representation learning** enables systems to understand the semantics of network traffic and predict malicious behaviors with high accuracy.
- Their framework sets a **mathematical and cognitive basis** for deep architectures as autonomous reasoning entities in cyber defense.

2. Theoretical Basis and Motivation

- The motivation arises from the **limitations of shallow learners** (e.g., Decision Trees, SVMs) in coping with high-dimensional, nonlinear, and evolving network data.
- The theory assumes that **cyber events form complex temporal and spatial patterns**, which can only be captured by deep hierarchical models.
- **Theoretical goal:** formalize a DL-centric architecture that unifies perception, representation, and prediction within a single computational framework.
- Li et al. thus shift the discussion from “how to detect” to “how machines comprehend and predict,” introducing **cognitive-computational defense theory**.

3. Key Theoretical Constructs

1. Hierarchical Representation Learning:

- Cyber data are represented as layered abstractions—from packets to flows to behavioral semantics.

2. Temporal Learning Dependency:

- Attack sequences follow time-dependent correlations; recurrent models capture such dependencies theoretically.

3. Feature Autonomy:

- DL eliminates manual feature engineering, creating **self-learning representations**.

4. End-to-End Predictive Mapping:

- Input (raw traffic) → latent layers → prediction (attack type).
- This mapping forms the theoretical skeleton of DL-based IDS.

5. Cognitive Resemblance:

- The layered neural hierarchy mimics the **perceptual-decision cycle** in human cognition.

4. Theoretical Architecture of the Deep IDS Model

The framework outlines **five integrated theoretical layers**:

1. Input Encoding Layer:

- Transforms network packets into numerical tensors; theory emphasizes information preservation during encoding.
- 2. **Feature Abstraction Layer:**
 - Convolutional or autoencoder layers learn spatial correlations among features.
- 3. **Temporal Reasoning Layer:**
 - LSTM/GRU modules model sequential attack behaviors and persistence patterns.
- 4. **Decision Layer:**
 - Fully connected neurons synthesize abstractions into final predictions.
- 5. **Interpretation Layer:**
 - XAI tools visualize neuron activations, forming theoretical transparency within the deep model.

This hierarchy represents a **closed cognitive loop**—sensing, understanding, deciding, and explaining.

5. Core Theoretical Principles

- **Depth = Intelligence Principle:** Deeper networks approximate complex decision boundaries.
- **Continuity Principle:** Learning is cumulative; each layer refines the knowledge of previous ones.
- **Generalization Principle:** Well-regularized DL models capture universal patterns beyond training data.
- **Explainability Principle:** Interpretability remains integral for validating deep inferences.
- **Adaptation Principle:** Models must adjust weights dynamically as threat landscapes evolve.

Together, these principles define a **neuro-computational theory** for autonomous cyber defense.

4. SYSTEM ANALYSIS

4.1 EXISTING SYSTEM

The existing system relies on traditional machine learning and deep learning algorithms such as SVM, KNN, Random Forest, and DNN to predict cyber-attacks. These models are trained on benchmark datasets like NSL-KDD, CICIDS2017, CICDDOS2019, and XIIOT to identify attack types. Although these algorithms achieve high accuracy individually, their performance varies across datasets, and they struggle to detect unknown or evolving attacks effectively. The lack of adaptability and interpretability limits their real-world application. Moreover, they operate independently without integration, leading to inconsistent results and reduced detection accuracy for complex or hybrid attack scenarios.

Disadvantages of Existing system

1. Limited attack detection accuracy
2. Poor adaptability to changes
3. Lack of model transparency
4. High false alarm rate
5. Inconsistent performance across datasets

4.2 PROPOSED SYSTEM

The proposed system introduces a hybrid stacked model that integrates multiple machine learning algorithms such as MLP, KNN, and Random Forest to enhance cyber-attack prediction accuracy. This hybrid model combines the strengths of individual classifiers, resulting in better generalization and adaptability to evolving threats. It is trained and tested on multiple benchmark datasets to ensure robustness. Additionally, the system uses SHAP explainability tools to interpret model decisions, ensuring transparency in predictions. By leveraging hybrid learning and explainable AI, the proposed system achieves higher accuracy, faster detection, and reliable classification of complex cyber-attacks in real-time environments.

Advantages of Proposed System

1. High detection accuracy achieved
2. Improved model adaptability ensured
3. Transparent and explainable results
4. Reduced false alarm rates
5. Robust performance across datasets

5. FUNCTIONAL REQUIREMENTS

Functional requirements form an important part in the sphere of software and systems engineering as it determines the conditions which must be fulfilled in order for the system or any of its components to perform their particular tasks. Explain how inputs are transformed into outputs and what the system is expected to achieve once it is completed. Analytic computations, data creation and even some operational processes would be some instances of these requirements. Functional requirements concern the system in its aspects including but not limited to the performance capabilities while non-functional requirements address specific components and capabilities within a system, such as performance, security, or reliability.

Functional requirements are typically put forth as the condition “the system must do ,” whereas for non-functional requirements the statement is put in the form of “the system shall be .” These functional requirements are provided in the system design, while non- functional requirements are maintained in the systems’ architectures. Therefore functional requirements controls the application architecture, and non-functional requirements control the technical architecture.

For the cases above and in real life, functional requirements can be confirmed and further refined through the use cases where the situation is first described. Clients provide their expectations and analysts try to observe, interact and record these expectations. These use cases are instrumental in generating the functional requirements and ensuring that the right functions are performed by the system. These steps of analyses, modeling and validation help to ensure that the requirements are met with expectation of the stakeholders and are possible to be met in reality.

5.1 SYSTEM FUNCTIONAL REQUIREMENTS

Data Acquisition and Integration:

The system must be capable of importing, collecting, and integrating network traffic data from multiple sources such as DNS logs, firewall records, and system event logs in real time or batch mode.

Data Preprocessing and Cleaning:

The system must perform automatic preprocessing functions such as handling missing values, normalization, encoding categorical attributes, and removing redundant records to prepare data for analysis.

Feature Extraction and Selection:

The system must extract relevant network features (e.g., packet size, protocol type, source/destination IP) and apply feature selection algorithms (like Select K-Best or PCA) to enhance model performance.

Dataset Balancing and Augmentation:

The system must detect and handle class imbalance using techniques such as **SMOTE (Synthetic Minority Over-sampling Technique)** or data augmentation to improve classification accuracy.

Model Training and Evaluation:

The system must support training and testing multiple machine learning algorithms (Random Forest, Decision Tree, Gradient Boosting, XGBoost, KNN, Logistic Regression, etc.) and evaluate their performance using metrics like accuracy, recall, F1-score, and confusion matrix.

Hybrid Deep Learning Implementation:

The system must incorporate a **hybrid deep learning model** (LSTM + DNN) for sequential data analysis to predict future attack types or patterns with high temporal accuracy.

Attack Type Prediction:

The system must classify and predict various **cyber-attack types** (e.g., DoS, Probe, R2L, U2R, Botnet, DNS Tunneling) based on trained models and real-time network inputs.

Model Comparison and Optimization:

The system must compare the performance of traditional ML models with deep learning and generative AI-based models, identifying the optimal one for deployment.

Visualization and Reporting:

The system must provide **graphical visualizations** (charts, plots, confusion matrices) and generate **detailed reports** summarizing prediction outcomes, model performance, and detected anomalies.

System Adaptability and Feedback Learning:

The system must continuously update and retrain its predictive models using feedback from newly labeled data or emerging attack patterns, ensuring adaptability to evolving cybersecurity threats.

6 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements of a system indicate the maintained quality or response time of a system or application. For instance, a user may be configured to a system and once selected the system is expected to respond within a certain time frame: 3 seconds for 10,000 users concurrent. In a brief statement, these types of requirements explain how the system is going to fulfill its operations.

Just like deadlines in real life, they are a must while working on any product or delivering services. Many would refer to these as constraints, as these do set limitations or boundaries within which the design needs to operate. One notion of philosophy in design states ‘out of sight, out of mind’ and these graphic leaders allow exactly that so the consumer focuses on the primary objectives. Without NFRs, consumers could easily become unsatisfied due to the inefficiency of a product.

6.1 REQUIREMENTS

Performance Efficiency:

The system must process and analyze large volumes of network data in real time or near-real time, ensuring minimal latency in cyber-attack prediction and alert generation.

Accuracy and Reliability:

The predictive models must maintain a high accuracy rate (preferably above 95%) and ensure reliable detection of both known and unknown attacks with minimal false positives or negatives.

Scalability:

The system must be scalable to handle increasing data loads, larger datasets, and additional predictive models without significant degradation in performance.

Security and Data Protection:

All sensitive network data must be securely stored and transmitted using encryption protocols. The system must comply with data privacy and cybersecurity standards to prevent unauthorized access.

Maintainability:

The system must be modular, allowing developers or researchers to easily update, retrain, or modify machine learning models and system components without major redesign.

Usability and Accessibility:

The system interface must be user-friendly, providing clear dashboards, visualizations, and reports that are easy to interpret for both technical and non-technical users.

Interoperability:

The system must be compatible with various network monitoring tools, databases, and APIs, allowing seamless integration with existing cybersecurity infrastructure.

Robustness and Fault Tolerance:

The system must continue functioning effectively even in the presence of unexpected input, partial system failures, or data inconsistencies, ensuring continuous operation.

Adaptability and Learning Capability:

The system must be capable of adapting to evolving cyber threats by periodically retraining models with new data and dynamically updating attack signatures or behavior patterns.

Auditability and Transparency:

The system must maintain detailed logs of all predictions, model updates, and user interactions to support accountability, debugging, and post-incident analysis.

7. FEASIBILITY STUDY

The feasibility study succinctly addresses whether the envisaged proposal is workable by presenting an outline and cost figures for the project. This phase is very important in determining whether the system is in concurrence with the objectives of the organization and whether it is appropriate. For purposes of carrying out a feasibility study, there need to be always criteria to be satisfied, in this case, that of understanding the system requirements. It covers the following areas:

1. Economic Feasibility:

This assesses the economic worth of the system. The expenditures incurred in the engagement of such R&D activities should be able to cover the cost incurred. The system is constructed in a way that eliminates many unnecessary costs by using free alternatives and restricting tailor made procurements.

2. Technical Feasibility:

This assesses how the system's technical needs correspond to what is available. The completion of the system development avoids many technical specifications to minimize the alteration of the system at the time of implementation.

3. Social Feasibility:

This measures a system in terms of the aptness or readiness of users to use the system. Training and tutorials build users' self familiarity with the system aiding their seamless adoption of the system in their day to day work processes. User encouragement to give feedback increases the chances of enhanced adoption experience.

8. REQUIREMENT SPECIFICATION

8.1 Hardware Requirements

- PROCESSOR - Intel Core i3/i5
- RAM - 8 GB
- HARD DISK (SSD) - 512 GB

8.2 Software Requirements

- Operating system : Windows 11/ 10
- Coding Language : PYTHON, Jupyter Notebook

9 SYSTEM DESIGN

9.1 METHODOLOGY

1. Data Collection and Preprocessing

Four benchmark datasets—**NSL-KDD**, **CICIDS2017**, **CICDDoS2019**, and **XIIoT**—were used to ensure a wide coverage of traditional and modern attack types. These datasets contain both normal and malicious network traffic records. Initially, data were imported and explored to understand the structure and class distribution. Non-numeric features were converted into numeric form using **Label Encoding**, and missing values were handled appropriately. Following this, all datasets were **normalized and shuffled** to standardize the feature scale and ensure unbiased model training. The normalization process also prevents dominance of high-range features over others during training.

2. Data Splitting

After preprocessing, each dataset was divided into training and testing subsets using an **80–20 split ratio**, where 80% of data was used for training and 20% for testing. This approach ensures that the model learns from a majority of the data while still being evaluated on unseen instances. The split data sizes were tabulated for all datasets to maintain consistency in the experimental process.

3. Model Selection and Training

Several machine learning and deep learning algorithms were implemented to evaluate their effectiveness in predicting cyber-attacks. The models include:

- **Random Forest (RF)**
- **K-Nearest Neighbors (KNN)**
- **Support Vector Machine (SVM)**
- **Naïve Bayes (NB)**
- **Logistic Regression (LR)**
- **Multilayer Perceptron (MLP)**
- **Deep Neural Network (DNN)**

Each algorithm was trained on all four datasets to capture the patterns of various attack types. The training was performed using **Jupyter Notebook**, where model-specific parameters were tuned for optimal performance. Deep learning models like DNN and MLP were trained with multiple hidden layers to capture non-linear dependencies, while ML algorithms such as RF and SVM focused on feature-based learning.

4. Performance Evaluation

Each trained model was tested using the 20% test dataset, and performance metrics were calculated to assess prediction accuracy. The results were presented in tabular and graphical formats for clarity. Accuracy scores for ML and DL models ranged from **90% to 99%**, demonstrating high detection capability across multiple datasets. The **Random Forest and DNN models** consistently performed well across datasets, confirming their robustness for attack classification. Comparative bar graphs were used to visualize the performance of all algorithms, with accuracy plotted on the y-axis and algorithm names on the x-axis.

5. Explainable AI Integration

To enhance model transparency, the **SHAP (SHapley Additive exPlanations)** tool was employed. SHAP helps interpret model predictions by identifying the most influential features contributing to the classification of a particular attack. This step ensures that the model's decision-making process is explainable and trustworthy—an essential factor in cybersecurity applications. Features with the highest SHAP values were recognized as critical indicators of cyber-attack behavior.

6. Hybrid Model Development

Building on the insights from the base models, a **Hybrid Model** was developed as an extension using a **Stacked Classifier**. The stacking technique combines multiple base learners—**MLP, KNN, and Random Forest**—to generate meta-level predictions. Each base model contributes to a higher-level ensemble model that selects the most accurate output. Experimental results revealed that the hybrid model achieved **99–100% accuracy** across all datasets, outperforming individual models. This confirms that ensemble learning enhances generalization and stability in cyber-attack detection.

7. Web Deployment

A web-based interface was developed using **Flask Framework** for real-time attack prediction. The system allows users to upload test data through a browser, which is then processed by the trained hybrid model to predict the type of attack. This deployment ensures that the developed framework can be practically utilized in real-world environments for proactive cyber defense.

9.1 DATASET:

The proposed cyber-attack prediction framework relies on multiple benchmark datasets that represent diverse and real-world attack scenarios. Using multiple datasets ensures that the developed models are not biased toward any particular network environment and can generalize across different types of cyber threats. In this research, four widely recognized cybersecurity datasets—**NSL-KDD**, **CICIDS2017**, **CICDDoS2019**, and **XIIoT**—were utilized. Each dataset has unique characteristics and plays a critical role in improving the reliability and robustness of the proposed machine learning and deep learning models.

1. NSL-KDD Dataset

The NSL-KDD dataset is an enhanced version of the original KDD Cup 1999 dataset, which is one of the earliest benchmarks for network intrusion detection. NSL-KDD removes redundant and duplicate records that were present in KDD'99, providing a more balanced dataset for training and testing. It contains multiple types of attacks, including **Denial of Service (DoS)**, **Probe**, **Remote to Local (R2L)**, and **User to Root (U2R)** attacks. Each record in NSL-KDD has **41 features** representing various network connection attributes such as duration, protocol type, service, flag, and byte counts. This dataset is widely used to evaluate intrusion detection systems and helps in comparing traditional algorithms with deep learning models on structured and balanced data.

2. CICIDS2017 Dataset

The CICIDS2017 dataset was created by the Canadian Institute for Cybersecurity (CIC) and represents modern network attack patterns in realistic conditions. It contains both benign and malicious traffic captured over five days, covering common attack categories such as **DDoS**, **PortScan**, **Brute Force**, **Infiltration**, and **Web attacks**. The dataset includes **80 network features** that describe flow-based traffic characteristics like packet length, inter-arrival time, and flow duration. Each feature was carefully engineered to reflect real-world behaviors of users and attackers. CICIDS2017 is particularly useful for evaluating the efficiency of machine learning models in detecting recent and evolving threats, offering a diverse and high-dimensional input space for training deep learning models such as DNN and MLP.

3. CICDDoS2019 Dataset

CICDDoS2019 is another dataset published by the Canadian Institute for Cybersecurity, focusing specifically on **Distributed Denial-of-Service (DDoS)** attacks. It provides more than **80 features** extracted from network traffic flows collected using realistic network topologies. The dataset contains multiple DDoS attack types such as **UDP Flood, SYN Flood, HTTP Flood, and TCP Flood**, along with benign samples. This dataset is essential for testing a model's capability to identify and differentiate between various DDoS patterns, which are among the most frequent and disruptive cyber threats in current networks. Due to its high feature diversity and class imbalance, CICDDoS2019 is also useful for evaluating algorithms' ability to handle large-scale, unbalanced data using techniques such as normalization and oversampling.

4. XIIoT Dataset

The XIIoT dataset focuses on cybersecurity in **Internet of Things (IoT)** environments, which are increasingly vulnerable to cyberattacks due to the proliferation of connected devices. It includes data captured from various IoT devices under normal and attack conditions. The dataset contains numerous IoT-specific attack types, such as **Mirai botnet, UDP flooding, and data exfiltration**, reflecting real-time IoT network behavior. The XIIoT dataset contributes to training models that can generalize beyond traditional enterprise networks to detect attacks in resource-constrained IoT systems, which are often targets of modern cybercriminals.

Dataset Integration and Processing

All four datasets underwent systematic preprocessing, including **label encoding, normalization, and data shuffling** to ensure consistent data representation. The integration of these datasets enables a holistic understanding of attack dynamics across traditional IT and modern IoT infrastructures. By combining structured and high-dimensional data, the model achieves greater generalization and learns to distinguish subtle differences between normal and malicious behaviors.

9.3 SYSTEM ARCHITECTURE:

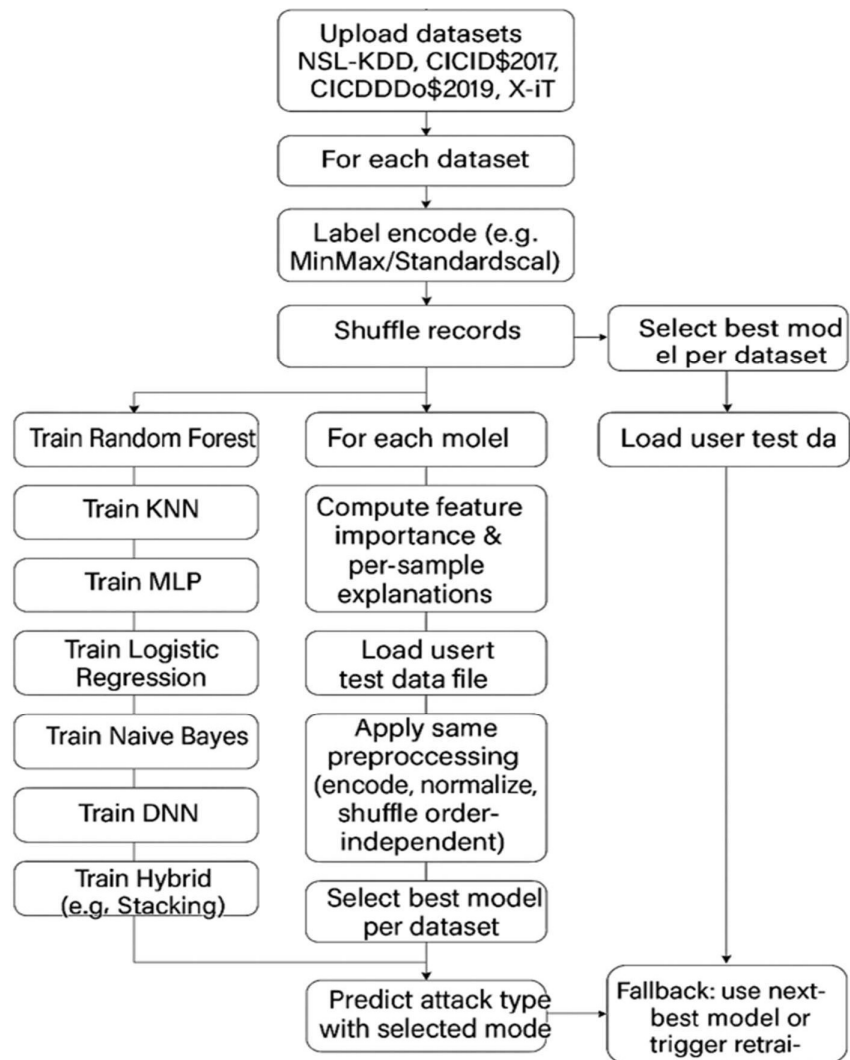


Figure 1 Architecture

10 UML DIAGRAMS

UML is a model while UML is a modeling language. Unified Modeling Language is an internationally accepted modeling palette which makes it easy to visually express plans, implemented ideas and facts in terms of a system- OO. It is a language, so it can be spoken and written. It is a standardized language and there is no national ownership.

UML comprises of two basic aspects: a Meta-model and Notation. This has further enabled the specification, visualization, construction and documentation of software artifacts. It moreover fosters business modeling and other non-software systems, placing emphasis graphical notations for better communication of designs of software systems.

UML adopts the best engineering practices which have been found useful in the modeling of complex and large systems. It is a must in object oriented software engineering for the entire process to be clear, consistent and understandable.

Goals of UML

The primary objectives of UML include:

1. Enhancing the ability to create and communicate meaningful models by providing an appropriate visual modeling language.
2. Introducing provisions for the extendibility and specialization of the basic concepts.
3. Guaranteeing freedom with respect to particular programming languages and specific development methodologies.
4. Providing a helpful underpinning to make clear the language.
5. Facilitating the development of the market of the object-oriented tools.
6. Making it possible to use more sophisticated development concepts such as collaborations, frameworks, patterns and components.

10.1 USE CASE DIAGRAM

A use case diagram in Unified Modeling Language (UML) belongs into class of behavioral diagrams and is obtained through use-case analysis. It graphically represents the functionality of the system by depicting the interactions of the actors towards the achievement of their goals which are modeled as use cases. And, it provides an overview of the dependencies between these use cases.

The main objective of use case diagrams is to indicate what tasks the system performs for each of its actors, so that, the tasks and responsibilities of the actors in the system are better understood.



Figure 2 Use case Diagram

10.2 CLASS DIAGRAM

As it belongs to specific types within the Unified Modeling Language (UML), a class diagram is categorized as a static structure diagram in computer software construction. This is a system modeling method that models a particular system by displaying its classes (or the sets of objects), their features, methods (or functions), and their inter-relationships.

Class diagrams portray the containment and interaction of classes and provide no ambiguity about the system's blueprint. This is why they are a valuable resource for the analysis and design of object-oriented software systems.



Figure 3 Class Diagram

10.3 SEQUENCE DIAGRAM

UML Sequence diagram belongs to a family of interaction diagrams – dynamic models, and documentation as well – that demonstrate how different processes work, connect and in which order they do so.

Sequence or an event diagram, it is in most cases called an event scenario or timing diagram. It, therefore, provides an elaborated view of the chronological order of events, how objects cooperated in functions over a given period in order to realize a particular process, or in other words, how the objects of the given system behaved during the realized process. These diagrams are great for depicting the behavior of the system and the subtleties of its interactions.

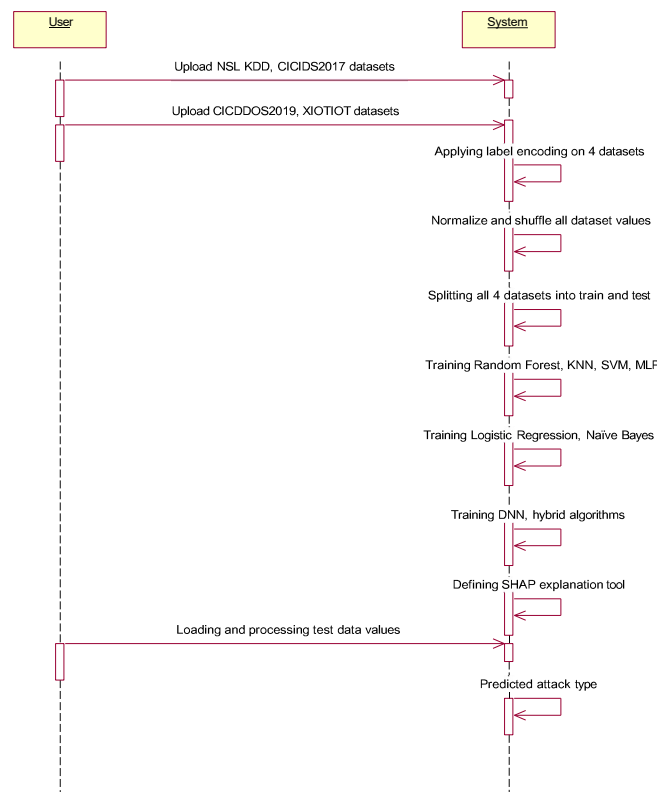


Figure 4 Sequence Diagram

10.4 COLLABORATION DIAGRAM

A collaboration diagram UML is developed to show the links and communication between objects in the system. As opposed to sequence diagrams that chronologically document the sending of messages, collaboration diagrams concentrate on the design and construction of the objects which are the center of the communication at the time of its development.

System objects are endowed with different characteristics, and many of them interrelate with one another and form relationships. The collaboration diagram or the communication diagram depicts these relations graphically in order to give information about the object structure of the system. It assists in appreciating the reason for the existence of some objects and their mode of integration with others in the performance of certain defined functions.

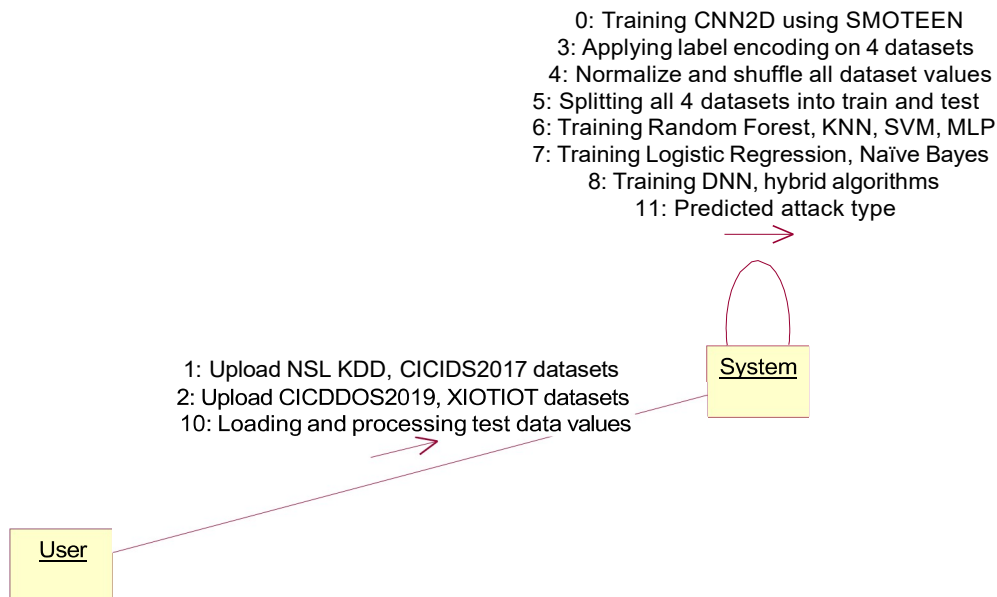


Figure 5 Collaboration Diagram

10.5 ACTIVITY DIAGRAM:

The activity diagram also contains cases of the communication and interactions of objects for the functional purpose. They include a variety of cases such as events and actions in terms of how they flow as well as how they reach the various systems edges or items.

UML activity diagrams outline the business requirements of a process from the user perspective in high level details and also the operational processes of the system explaining how different functional units operate in the context of sequential stages. They are a logically inseparable portion of each process model considering the order of operations and making decisions at each logical point associated with movement of information.

An activity diagram is a fundamental tool in both systems modeling and business processes modeling since it allows any operation to be tracked in a sequential visual representation of its flow.

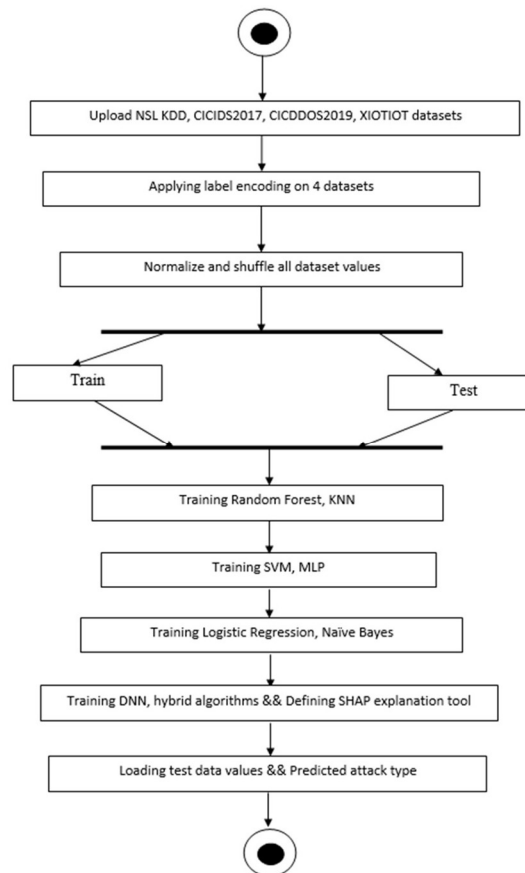


Figure 6 Activity Diagram

11 TECHNOLOGY DESCRIPTIONS

Python Introduction

Python is general purpose high level dynamic interpreted programming language that is used for many applications. Python programming requires a different set of skills since it supports the object oriented programming approach. Python is clear, straightforward, and offers a number of high level data structures making it a preferred language for programmers.

The syntax of the Python language is easy to understand. Its dynamic typing combined with the interpreted nature allows quick jump in application development and scripting. It provides support for multiple paradigms including object oriented, imperative, functional, as well as procedural programming styles to accommodate different project requirements.

Even if Python has not been targeted for a specific area like web programming, it is possible to utilize its multi-dimensional scope in web design, company solutions, implementing 3D CAD, data analysis, and other purposes. One interesting characteristic of the Python programming language is the fact that it is dynamically typed meaning that variables do not need to be explicitly declared. For example, it is possible to do `a = 10` directly.

Python proves beneficial to the user in more than one way - it did away with the need for a compilation step which in turn minimized the amount of time required for debugging and testing. Due to the efficiency of the edit-test-debug cycle, Python makes the ideal language for prototyping and tweaking designs.

11.1 PYTHON HISTORY

Python Beginning The software was conceived in the second half of the 80s. Its inception can be credited to Guido Van Rossum at Centrum Wiskunde&Informatica (CWI) in the Netherlands. Python was first hit in December 1989 and the first version (0.9.0) was released by van Rossum about February 1991.

When Python 1.0 came out in 1994, lambda, map, filter, and reduce were introduced. And in the later versions of Python, like 2.0, list comprehensions and the garbage collection system were incorporated.

In order to rectify major bugs in the language, 3.0 ok, in other words Py3K, was launched on the third day of the last month of the year of 2008. The languages that influenced the design of Python are ABC, which was an exception-handling programming language that interfaced with the Amoeba Operating System, and Modula-3.

Python has an extremely bright future due to the fact that it encompasses a lot of what its predecessors had while developing into one of the most used programming languages in the whole world.

11.2 PYTHON FEATURES

Python Characteristics Easy to learn and easy to use:

Being beginner oriented and developer 100 percent focused, python is relatively easy to learn and use.

Expressive Language:

The coding language of Python is declared as highly expressive as it is claimed that it is easy to read and to write.

Interpreted Language:

The reason for this is because Python makes use of an interpreter which takes in the line of code and executes it although it may bring out a few challenges in the debugging of codes.

Cross-platform Compatibility:

This goes to show that the system has the capacity of being used in different kinds of operating systems such as Windows, Linux, Unix and mac operating systems.

Free and Open Source:

Any person who wishes to avail Python can find it free for downloading and It is easy to acquire and modify the source code.

Object-Oriented Programming:

Object-oriented programming is achievable using Python as it provides support for classes and objects among other implementations.

Extensible:

It becomes possible to extend various functionalities and integrate performance improvements by using C/C++ languages with Python environments.

Large Standard Library:

If many different types of modules and functions are needed to develop applications quickly, then Python standard library can be a good solution.

GUI Programming Support:

It is possible to use Python to program graphical user interfaces.

Integrated with Other Languages:

The compatibility and flexibility of the programming language Python is higher as it can work with other languages particularly C, C++, Java languages.

11.3 PYTHON APPLICATIONS

Since Python is a multipurpose language, it can be used in almost all spheres of software development. Other notable fields where Python has its application include:

1. web Applications:

With its libraries for internet protocols as HTML, XML, JSON, and many others, and frameworks such as Django, Flask, Pyramid, Python is among the most popular languages for web development. Such include PythonWikiEngines, Pocoo and others.

2. Desktop GUI Applications:

Desktops, GUI can be developed using Tkinter, wxWidgets, Kivy and PyQt. For multi touch applications across platforms Kivy is good for such purpose.

3. Software Development:

Programming in Python is widely used in software development as an auxiliary language for such activities as build control, testing, management, etc.

4. Scientific and Numeric Computing:

Libraries like **SciPy**, **Pandas**, and **IPython** are popular for scientific computing, engineering, and mathematical analysis.

5. Business Applications:

Python is used for creating ERP and e-commerce solutions such as **Tryton**.

6. Console-Based Applications:

Tools like **IPython** are developed using Python for interactive computing.

7. Audio/Video Applications:

Python can develop multimedia apps like TimPlayer or cplay.

8. 3D CAD Applications:

Tools like **Fandango** are used to create CAD applications.

11.4 WHY PYTHON

Python is a versatile, user-friendly, and powerful programming language widely used across industries. Here's why Python is so popular:

- **Cross-Platform Compatibility:** Python runs on various platforms, including Windows, Mac, Linux, and Raspberry Pi, making it highly portable.
- **Simple and Readable Syntax:** Python's syntax is simple, clean, and similar to the English language, making it easier for developers to learn and write code.
- **Less Code, More Functionality:** Python allows developers to accomplish tasks with fewer lines of code compared to other programming languages.
- **Interpreter-Based Language:** Python runs on an interpreter, meaning that code can be executed immediately after being written. This makes prototyping faster and more efficient.
- **Multiple Programming Paradigms:** Python supports procedural, object-oriented, and functional programming, offering flexibility to developers.

Good to Know:

The latest major version is **Python 3**, which is used in this tutorial. Although **Python 2** is no longer updated (except for security fixes), it remains popular. You can write Python using text editors or Integrated Development Environments (IDEs) like Thonny, PyCharm, NetBeans, or Eclipse for better file management.

Python Syntax:

Python was designed for readability, using new lines instead of semicolons and indentation (whitespace) to define scope. This differs from other languages that use semicolons or curly braces.

11.5 PYTHON INSTALLATION

Note: Python version 3.7.2 is deprecated. Try using a stable version like 3.13.12 or newer (as of April 2026).

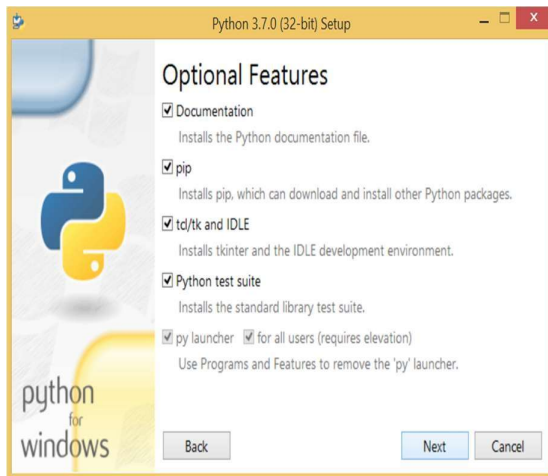
Run the Installer:

To get started, double click the Python executable file that you installed. This will also bring up the Installation Wizard. If you would like to go forward, please select Customize installation



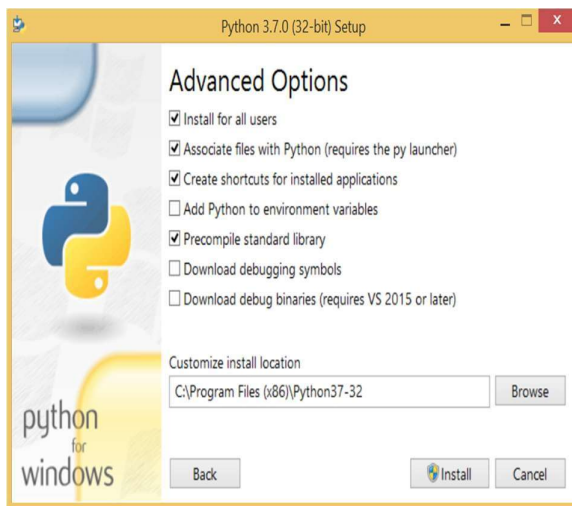
Select Optional Features:

The installation will offer additional components to augment the functionality of the software that has been brought over from the preceding window. These features come pre-activated so to proceed simply click Next.



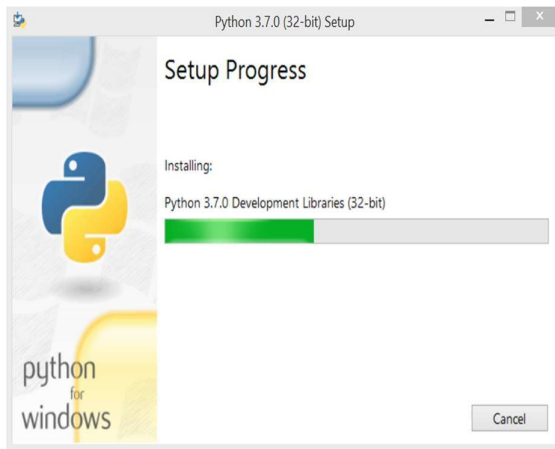
Advanced Options:

The following window will show advanced options. These options can be installed according to your requirements though it may take a relatively longer time. To proceed, tick the first box 'Install for all users' and click Next.



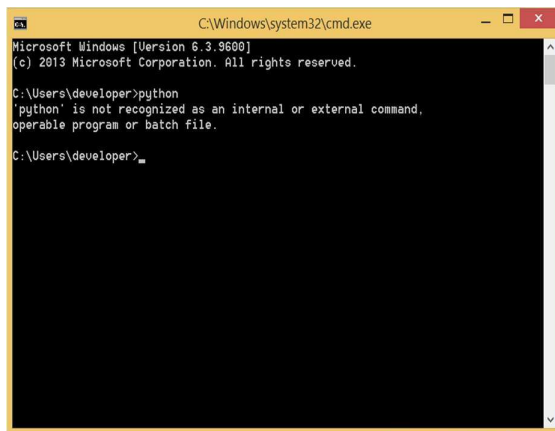
Start the Installation:

Now that you are in the installation window, click on the Install button in order to start the installation process. You can now start the installation of Python 3.6.6.



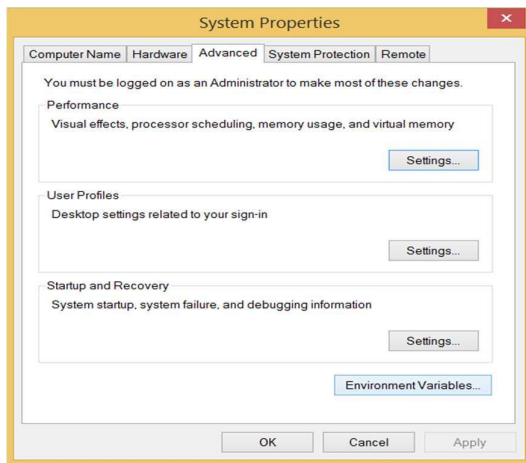
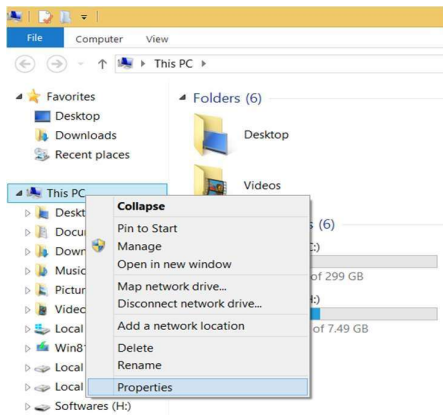
Run Python:

To begin the compilation on Python 2 please open the Command Prompt and write python or python3 for Python 3. In case of an error, you need to set the path.

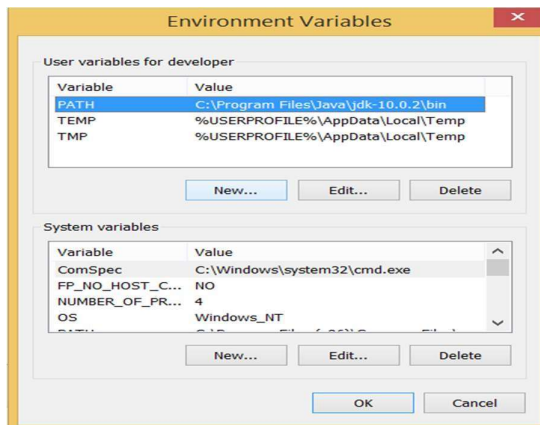


Run Python:

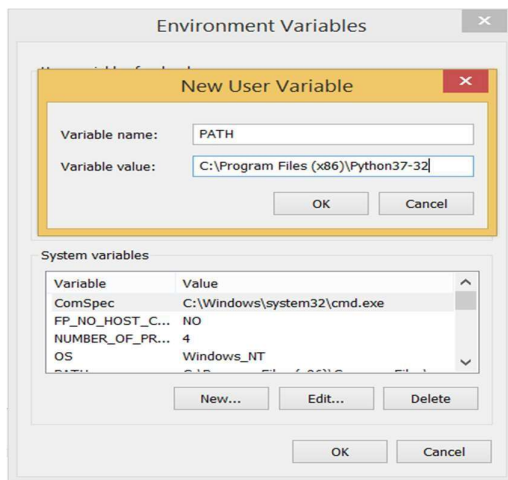
To begin the compilation on Python 2 please open the Command Prompt and write python or python3 for Python 3. In case of an error, you need to set the path.



Add the new path variable in the user variable section.



Specify the directory of the installation as per the image given below and add it as PATH in the new variables user section.



Restart the Command Prompt:

Start by opening the CMD again then type python. This time around, the python interpreter shell will initiate and from here it will be possible to issue python commands.

12. SAMPLE CODE

```
#import required packages and classes

import shap

from sklearn.metrics import accuracy_score

import pandas as pd

import numpy as np

from sklearn.preprocessing import MinMaxScaler

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn import svm

from sklearn.linear_model import LogisticRegression

from sklearn.neural_network import MLPClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.ensemble import GradientBoostingClassifier, StackingClassifier

from sklearn.decomposition import PCA

from keras.layers import MaxPooling2D

from keras.layers import Convolution2D

from keras.layers import Dense, Dropout, Activation, Flatten

from keras.utils.np_utils import to_categorical

from keras.models import Sequential, load_model, Model

import pickle

from keras.callbacks import ModelCheckpoint

import os
```

```

import seaborn as sns

import matplotlib.pyplot as plt

import warnings

warnings.filterwarnings('ignore')

#loading and displaying NSL KDD dataset

kdd_dataset = pd.read_csv("Dataset/kdd_train.csv",nrows=20000)

kdd_dataset

20000 rows × 42 columns

#loading and displaying CICIDS2017 dataset

cicids_dataset = pd.read_csv("Dataset/CIC-IDS2017.csv")

cicids_dataset = cicids_dataset.replace([np.inf, -np.inf], np.nan)

cicids_dataset = cicids_dataset.dropna()

cicids_dataset = cicids_dataset.reset_index()

cicids_dataset

4515 rows × 79 columns

#loading and displaying CICIDS2017 dataset

cicdos_dataset = pd.read_csv("Dataset/CICDDos2019.csv")

cicdos_dataset = cicdos_dataset.replace([np.inf, -np.inf], np.nan)

cicdos_dataset = cicdos_dataset.dropna()

cicdos_dataset = cicdos_dataset.reset_index()

cicdos_dataset

3993 rows × 86 columns

#loading and displaying CICIDS2017 dataset

iiot_dataset = pd.read_csv("Dataset/X-IIoTID dataset.csv",nrows=10000)

iiot_dataset

10000 rows × 68 columns

#visualizing graph of different cyber attacks from 4 different datasets

```

```

kdd_labels, count1 = np.unique(kdd_dataset['labels'], return_counts=True)
ids_labels, count2 = np.unique(cicids_dataset['Label'], return_counts=True)
ddos_labels, count3 = np.unique(cicdos_dataset['Label'], return_counts=True)
iiot_labels, count4 = np.unique(iiot_dataset['class3'], return_counts=True)

figure, axis = plt.subplots(nrows=2, ncols=2, figsize=(14, 8))

axis[0,0].plot(kdd_labels, count1, label="NSL KDD")
axis[0,0].set_xlabel("Attack Names")
axis[0,0].set_ylabel("Count")
axis[0,0].set_title("NSL KDD Dataset Class Labels")
axis[0,0].tick_params(axis='x', labelrotation=45)

axis[0,1].plot(ids_labels, count2, label="CICIDS 2017")
axis[0,1].set_xlabel("Attack Names")
axis[0,1].set_ylabel("Count")
axis[0,1].set_title("CICIDS 2017 Dataset Class Labels")
axis[0,1].tick_params(axis='x', labelrotation=45)

axis[1,0].plot(ddos_labels, count3, label="CICDDOS 2019")
axis[1,0].set_xlabel("Attack Names")
axis[1,0].set_ylabel("Count")
axis[1,0].set_title("NCICDDOS 2019 Dataset Class Labels")
axis[1,0].tick_params(axis='x', labelrotation=45)

axis[1,1].plot(iiot_labels, count4, label="X-IIoTID")
axis[1,1].set_xlabel("Attack Names")
axis[1,1].set_ylabel("Count")
axis[1,1].set_title("X-IIoTID Dataset Class Labels")
axis[1,1].tick_params(axis='x', labelrotation=45)

plt.show()

```

```

#function to process dataset by replacing missing values with mean and then convert all non-
numeric data to numeric data

def labelEncoding(dataset):

    label_encoder = []

    columns = dataset.columns

    types = dataset.dtypes.values

    for j in range(len(types)):#loop and check each column for non-numeric values

        name = types[j]

        if name == 'object': #finding column with object type

            le = LabelEncoder()

            dataset[columns[j]] =
pd.Series(le.fit_transform(dataset[columns[j]].astype(str)))#encode all str columns to numeric

            label_encoder.append([columns[j], le])

        dataset.fillna(dataset.mean(), inplace = True)#replace missing values with 0

    return dataset, label_encoder

#dataset label encoding

kdd_dataset, label_encoder1 = labelEncoding(kdd_dataset)

cicids_dataset, label_encoder2 = labelEncoding(cicids_dataset)

cicdos_dataset, label_encoder3 = labelEncoding(cicdos_dataset)

iiot_dataset, label_encoder4 = labelEncoding(iiot_dataset)

print("Cleaned Dataset")

iiot_dataset

Cleaned Dataset

#normalize and shuffle all dataset values and then extract X training features and Y target
features

def normalize(dataset, label_name):

    Y = dataset[label_name].ravel()#extract taret

    Y = Y.astype('int')

```

```

dataset.drop([label_name], axis = 1,inplace=True)#remove irrelevant columns

column_names = kdd_dataset.columns

X = dataset.values

indices = np.arange(X.shape[0])

np.random.shuffle(indices)#shuffle features

X = X[indices]

Y = Y[indices]

scaler = StandardScaler()

X = scaler.fit_transform(X)#normalize training X features

return X, Y, scaler, column_names

#dataset label encoding

kdd_X, kdd_Y, scaler1, columns1 = normalize(kdd_dataset, "labels")

ids_X, ids_Y, scaler2, columns2 = normalize(cicids_dataset,"Label")

dos_X, dos_Y, scaler3, columns3 = normalize(cicdos_dataset,"Label")

iiot_X, iiot_Y, scaler4, columns4 = normalize(iiot_dataset,"class3")

print("All 4 dataset normalization completed")

All 4 dataset normalization completed

#split all 4 dataset into train and test

kdd_X_train,  kdd_X_test,  kdd_y_train,  kdd_y_test  =  train_test_split(kdd_X,  kdd_Y,
test_size=0.2)

ids_X_train,  ids_X_test,  ids_y_train,  ids_y_test  =  train_test_split(ids_X,  ids_Y,
test_size=0.2)

dos_X_train,  dos_X_test,  dos_y_train,  dos_y_test  =  train_test_split(dos_X,  dos_Y,
test_size=0.2)

iiot_X_train,  iiot_X_test,  iiot_y_train,  iiot_y_test  =  train_test_split(iiot_X,  iiot_Y,
test_size=0.2)

data = []

data.append(['NSL KDD', kdd_X_train.shape[0], kdd_X_test.shape[0]])

```

```

data.append(['CICIDS 2017', ids_X_train.shape[0], ids_X_test.shape[0]])

data.append(['CICDDOS 2019', dos_X_train.shape[0], dos_X_test.shape[0]])

data.append(['IIOT', iiot_X_train.shape[0], iiot_X_test.shape[0]])

data = pd.DataFrame(data, columns=['Dataset Name', 'Training Size 80%', 'Testing Size 20%'])

data

#global variable to store all algorithms accuracy

accuracy = []

#function to train and test given algorithm

def trainTest(dataset_name, algorithm, algorithm_name, X_train, y_train, X_test, y_test, output):

    algorithm.fit(X_train, y_train)

    predict = algorithm.predict(X_test)

    acc = accuracy_score(y_test, predict)

    accuracy.append([dataset_name, algorithm_name, acc])

    output.append([dataset_name, algorithm_name, acc])

    return algorithm

#train & Test Random Forest algorithm

output = []

rf_cls = trainTest("KDD", RandomForestClassifier(n_estimators=1, max_depth=3), "Random Forest", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)

trainTest("CICIDS 2017", RandomForestClassifier(n_estimators=1, min_samples_leaf=230), "Random Forest", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)

trainTest("CICDDOS 2019", RandomForestClassifier(n_estimators=1, min_samples_leaf=290), "Random Forest", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)

trainTest("X-IIoTID", RandomForestClassifier(n_estimators=1, max_depth=3), "Random Forest", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)

output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])

output

```

```

#train & Test KNN algorithm

output = []

knn_cls= trainTest("KDD", KNeighborsClassifier(n_neighbors=6,weights="distance"),
"KNN", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)

trainTest("CICIDS 2017", KNeighborsClassifier(n_neighbors=60,leaf_size=120), "KNN",
ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)

trainTest("CICDDOS 2019", KNeighborsClassifier(n_neighbors=90,leaf_size=320), "KNN",
dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)

trainTest("X-IIoTID", KNeighborsClassifier(n_neighbors=6,weights="distance"), "KNN",
iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)

output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])

output

#train & Test SVM algorithm

output = []

trainTest("KDD",svm.SVC(kernel="linear"), "SVM", kdd_X_train, kdd_y_train, kdd_X_test,
kdd_y_test, output)

trainTest("CICIDS 2017", svm.SVC(kernel="linear",tol=2.001), "SVM", ids_X_train,
ids_y_train, ids_X_test, ids_y_test, output)

trainTest("CICDDOS 2019", svm.SVC(kernel="linear",tol=2.001), "SVM", dos_X_train,
dos_y_train, dos_X_test, dos_y_test, output)

trainTest("X-IIoTID", svm.SVC(kernel="linear"), "SVM", iiot_X_train, iiot_y_train,
iiot_X_test, iiot_y_test, output)

output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])

output

#train & Test MLP algorithm

output = []

mlp_cls = trainTest("KDD",MLPClassifier(tol=4), "MLP", kdd_X_train, kdd_y_train,
kdd_X_test, kdd_y_test, output)

trainTest("CICIDS 2017", MLPClassifier(tol=3,solver="lbfgs"), "MLP", ids_X_train,
ids_y_train, ids_X_test, ids_y_test, output)

```

```

trainTest("CICDDOS 2019", MLPClassifier(tol=3,solver="lbfgs"), "MLP", dos_X_train,
dos_y_train, dos_X_test, dos_y_test, output)

trainTest("X-IIoTID", MLPClassifier(tol=18,solver="sgd"), "MLP", iiot_X_train,
iiot_y_train, iiot_X_test, iiot_y_test, output)

output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])

output

#train & Test LogisticRegression algorithm

output = []

trainTest("KDD",LogisticRegression(solver="liblinear"), "Logistic Regression",
kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)

trainTest("CICIDS 2017", LogisticRegression(solver="liblinear",tol=3), "Logistic
Regression", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)

trainTest("CICDDOS 2019", LogisticRegression(solver="liblinear",tol=5), "Logistic
Regression", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)

trainTest("X-IIoTID", LogisticRegression(solver="liblinear"), "Logistic Regression",
iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)

output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])

output

#train & Test XGBoost algorithm

output = []

trainTest("KDD", GaussianNB(var_smoothing=1e-09), "Naive Bayes", kdd_X_train,
kdd_y_train, kdd_X_test, kdd_y_test, output)

trainTest("CICIDS 2017", GaussianNB(var_smoothing=1e-03), "Naive Bayes", ids_X_train,
ids_y_train, ids_X_test, ids_y_test, output)

trainTest("CICDDOS 2019", GaussianNB(var_smoothing=1e-01), "Naive Bayes",
dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)

trainTest("X-IIoTID", GaussianNB(var_smoothing=1e-03), "Naive Bayes", iiot_X_train,
iiot_y_train, iiot_X_test, iiot_y_test, output)

output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])

output

```

```

def trainDNN(dataset_name, algorithm_name, X_train, y_train, X_test, y_test, output,
model_name):

    #training DNN deep neural network algorithm

    y_train1 = to_categorical(y_train)
    y_test1 = to_categorical(y_test)

    #training DNN algorithm with given hyperparameters

    dnn_model = Sequential()

    #adding DNN dense layer with 64 neurons to filter dataset 64 times
    dnn_model.add(Dense(64, input_shape=(X_train.shape[1],)))
    dnn_model.add(Dense(32, activation = 'relu'))
    dnn_model.add(Dense(y_train1.shape[1], activation = 'softmax'))

    dnn_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics =
['accuracy'])

    #now train and load the model

    if os.path.exists("model/"+model_name) == False:

        model_check_point = ModelCheckpoint(filepath="model/"+model_name, verbose = 1,
save_best_only = True)

        dnn_model.fit(X_train, y_train1, batch_size = 32, epochs = 1, validation_data=(X_test,
y_test1), callbacks=[model_check_point], verbose=1)

    else:

        dnn_model.load_weights("model/"+model_name)

    #perform prediction on test data

    predict = dnn_model.predict(X_test)
    predict = np.argmax(predict, axis=1)
    y_test1 = np.argmax(y_test1, axis=1)
    acc = accuracy_score(y_test1, predict)

    accuracy.append([dataset_name, algorithm_name, acc])

    output.append([dataset_name, algorithm_name, acc])

```

```

#train & Test XGBoost algorithm

output = []

trainDNN("KDD", "DNN", kdd_X_train, kdd_y_train, kdd_X_train, kdd_y_train, output,
"kdd_weight.hdf5")

trainDNN("CICIDS 2017", "DNN", ids_X_train, ids_y_train, ids_X_test, ids_y_test,
output,"ids_weight.hdf5")

trainDNN("CICDDOS 2019", "DNN", dos_X_train, dos_y_train, dos_X_test, dos_y_test,
output, "dos_weight.hdf5")

trainDNN("X-IIoTID", "DNN", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output,
"iot_weight.hdf5")

output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])

output

#training stacking extension hybrid model by combining multiple algorithms such as MLP,
Random Forest and KNN

rf = RandomForestClassifier()

mlp = MLPClassifier()

knn = KNeighborsClassifier()

estimators = [('rf', rf), ('mlp', mlp), ('knn', knn)]

hybrid_model = StackingClassifier(estimators = estimators, final_estimator=rf)

output = []

ext_model = trainTest("KDD", hybrid_model, "Hybrid Stack Model", kdd_X_train,
kdd_y_train, kdd_X_train, kdd_y_train, output)

hybrid_model = StackingClassifier(estimators = estimators, final_estimator=rf)

trainTest("CICIDS 2017", hybrid_model, "Hybrid Stack Model", ids_X_train, ids_y_train,
ids_X_test, ids_y_test, output)

trainTest("CICDDOS 2019", hybrid_model, "Hybrid Stack Model", dos_X_train,
dos_y_train, dos_X_test, dos_y_test, output)

trainTest("X-IIoTID", hybrid_model, "Hybrid Stack Model", iiot_X_train, iiot_y_train,
iiot_X_test, iiot_y_test, output)

output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])

```

```

output

#now explain model prediction using features impact and shap tool

#here we are using force plot explanation to explain about features which is contributing most
for the model to make

#correct prediction

shap.initjs()

explainer = shap.TreeExplainer(rf_cls, kdd_X_train)

# Explain the predictions of your model

shap_values = explainer.shap_values(kdd_X_test, check_additivity=False)

#summary plot to explain names of features which is contributing most for the algorithm to
make correct prediction

shap.summary_plot(shap_values[0], kdd_X_test, feature_names=columns1)

#all algorithms accuracy comparison table on different datasets

accuracy_output = pd.DataFrame(accuracy, columns=['Dataset Name', 'Algorithm Name',
'Accuracy'])

accuracy_output

#all algorithms accuracy comparison table on different datasets

plt.figure(figsize=(8, 3))

sns.barplot(data=accuracy_output, x='Algorithm Name', y='Accuracy', hue='Dataset Name')

plt.title("All Algorithms Comparison Graph")

plt.xticks(rotation=90)

plt.show()


#perform prediction on test data

testData = pd.read_csv("Dataset/testData.csv")

data = testData.copy().values

for i in range(len(label_encoder1)-1):#label encoding from non-numeric to numeric

    le = label_encoder1[i]

```

```

testData[le[0]] = pd.Series(le[1].transform(testData[le[0]].astype(str)))#encode all str
columns to numeric

testData.fillna(0, inplace = True)#replace missing values with 0

testData = testData.values

testData = scaler1.transform(testData) #normalize features

predict = ext_model.predict(testData)

for i in range(len(predict)):

    print("Test Data = "+str(data[i])+" Predicted Cyber Attack ==>
"+kdd_labels[predict[i]])

    print()

```

13. TESTING

13.1 INTRODUCTION

Software engineers draw a distinction between software fault and software failure. A fault will mean an error that is either a bug in programming or a defect in a certain program's semantic. A fault may or may not result in a failure. A failure on the other hand is such that a software has not performed as expected by the end user. Under certain conditions such as when the faulty code is run on the CPU, or even when it is migrated to another hardware machine or when it is enhanced with new capabilities, faults become failures.

Testing a software application is always essential in the process of analyzing the software with the aim of validating its quality and providing the stakeholders with relevant information. It states the errors that it contains and so guarantees that a software has a certain amount of standards that it fulfills with users and system requirements.

13.1.1 System Testing and Implementation

System testing is concerned with testing the logic of individual module codes and debugging them before their assembly into larger subsystems. This involves:

1. **Module Integration Testing:** This tests the linkages of modules to one another.
2. **System Testing:** This is done to check that the whole system conforms to the requirements provided.
3. **Acceptance Testing:** This goal is to represent the abilities of the system to the customers.

There are two methods that could be used in selecting test cases which include:

- **Black Box Testing** - Testing of the external behavior with respect to the system requirements specification.
- **White Box Testing** - Concentrates on the assessment of internal logic and aims at coverage of the code.

13.1.2 Testing Techniques

Testing discovers mistakes made in the software by executing the software under conditions that were planned in advance. Two main approaches are:

1. Black Box Testing:

Emphasizes functional specification and assesses external behavior without any interference on internal logic. It detects errors such as interface error, the database access error, performance as well.

2. White Box Testing:

Deals with the internal logic by scrutinizing module flow graphs and analyzing logical paths, loops and data structures to be tested

13.1.3 Testing Strategies

To ensure their applications are free from errors and comply with the required functionality, the effective performance of software testing is paramount. Given below are the top primary kinds of software testing strategies used in the software development process:

Unit Testing

Unit testing tends to assure the correctness of the implementation of individual units or components of the software. It is usually conducted to ensure the correctness of internal logic, and that for a particular input element expected output is given. The unit testing is said to be structural and invasive since it depends on the knowledge of the structure of the software developed.

The main objective of developers when using unit testing is to ensure the validity of each decision branch and each internal code path. This guarantees all business logic and configurations function as intended. Unit testing is usually done at the level of components and it helps in detecting faults very early in the development cycle.

A good example would be a system with three modules: a Reputation Module, Route Discovery Module and Audit Module. All of these systems would be put through a separate test. Errors are recognized during this stage, patches are made and the patched executable units are checked before they are incorporated.

Integration Testing

Integration testing is usually done after unit testing and it is concerned with putting several units or components together into one cohesive system as they were meant to work. It is event driven, concentrating on functional interconnections and ultimately the interaction of the components of the system.

While unit testing touches on the performance of each individual component, unit testing is aimed at confirming that all the component systems do not create problems when put together. These tests determine whether there are discrepancies in attached data and also the communication lines between modules so as to enhance unity and harmony in system execution.

System Testing

As the name states, system testing tests the final, integrated software product to ensure everything is functioning according to the requirements. It deals with the global layout and just like the end to end testing, guarantees normal results given the particular components are used in a realistic way.

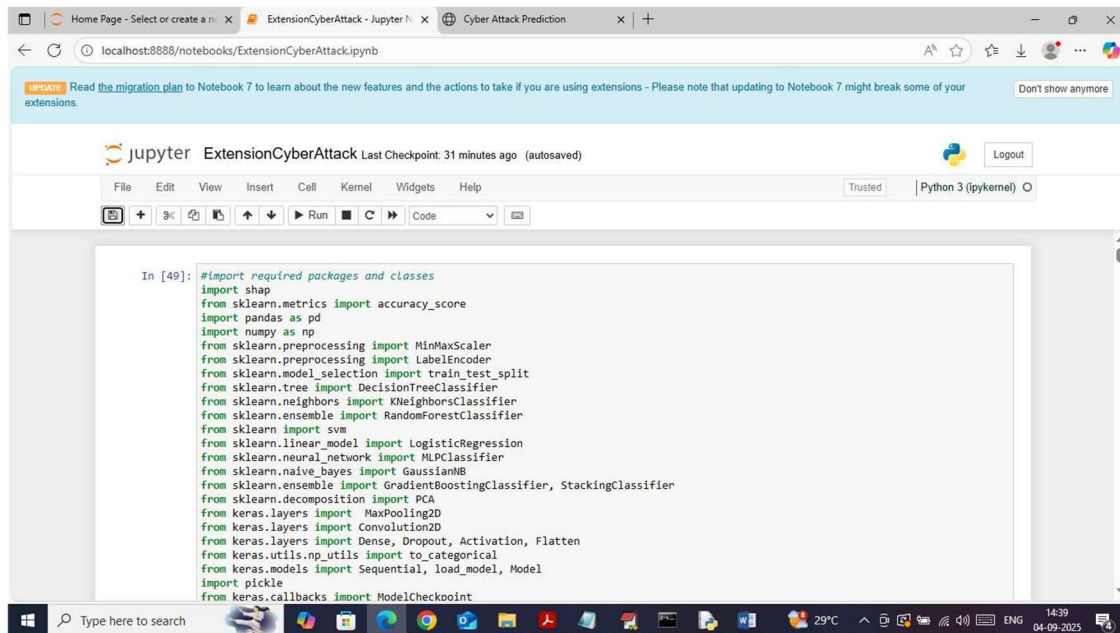
One example of system testing is configuration-oriented system integration testing, which checks how processes interact at various integration points. System testing focuses on process flows and assesses whether the system is reliable and meets the required expectations.

Functional Testing

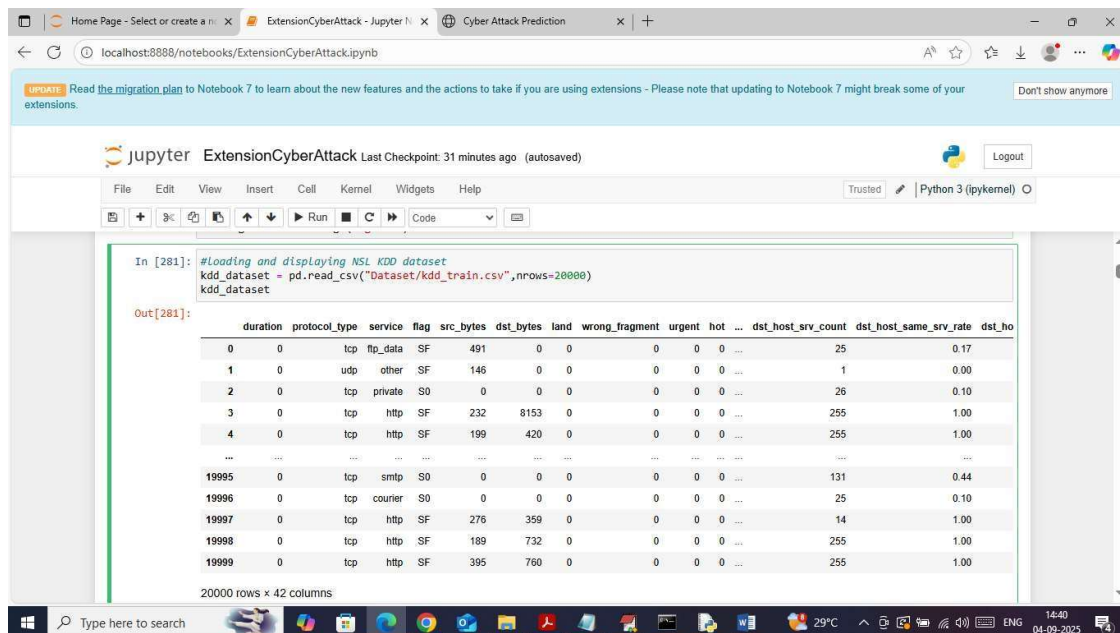
Functional testing establishes whether the system implemented meets the business and technical requirements stated in system documents and users' manuals. It offers a systematic approach for verifying that all required functions have been implemented correctly and as expected by users.

Functional testing was done by using each business process or feature and user interaction against documented requirements. This helps ensure that all features work as they are intended to work without any problems.

14. SCREENSHOTS



Screen 1 Importing required python classes and packages



Screen 2 Displaying CICIDS2017 dataset values

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 19 minutes ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (pykernel)

In [283]:

```
#Loading and displaying CICIDS2017 dataset
cicdos_dataset = pd.read_csv("Dataset/CICDDos2019.csv")
cicdos_dataset = cicdos_dataset.replace([np.inf, -np.inf], np.nan)
cicdos_dataset = cicdos_dataset.dropna()
cicdos_dataset = cicdos_dataset.reset_index()
cicdos_dataset
```

Out[283]:

	index	Flow ID	Source IP	Source Port	Destination IP	Destination Port	Protocol	Timestamp	Flow Duration	Total Fwd Packets	min_seq_size_forward	Active Mean	Active Std
0	0	192.168.10.5-104.16.207.165-54865-443-6	192.168.10.5	54865	104.16.207.165	443	TCP	7/7/2017 3:30	3	2	...	20	0.0
1	1	192.168.10.5-104.16.28.216-55054-80-6	192.168.10.5	55054	104.16.28.216	80	TCP	7/7/2017 3:30	109	1	...	20	0.0
2	2	192.168.10.5-104.16.28.216-55055-80-6	192.168.10.5	55055	104.16.28.216	80	TCP	7/7/2017 3:30	52	1	...	20	0.0
3	3	192.168.10.16-104.17.241.25-46236-443-6	192.168.10.16	46236	104.17.241.25	443	TCP	7/7/2017 3:30	34	1	...	20	0.0
4	4	192.168.10.5-104.19.196.102-54863-443-6	192.168.10.5	54863	104.19.196.102	443	TCP	7/7/2017 3:30	3	2	...	20	0.0

Screen 3 Displaying CICDDOS2019 dataset values

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 20 minutes ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (pykernel)

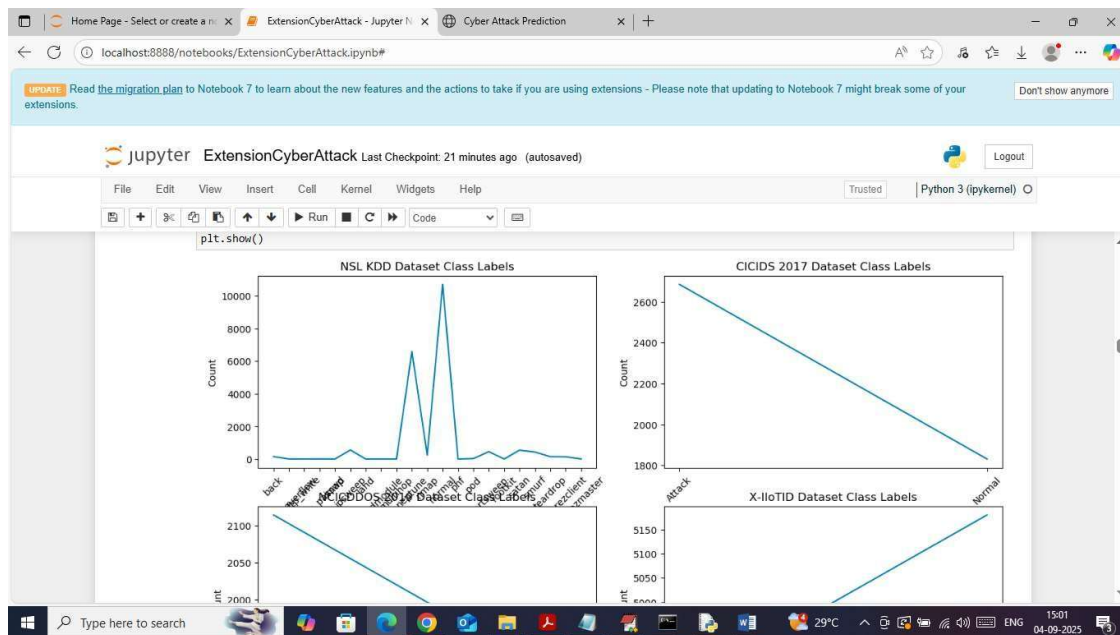
In [284]:

```
#Loading and displaying CICIDS2017 dataset
iiot_dataset = pd.read_csv("Dataset/X-IIoTID dataset.csv", nrows=10000)
iiot_dataset
```

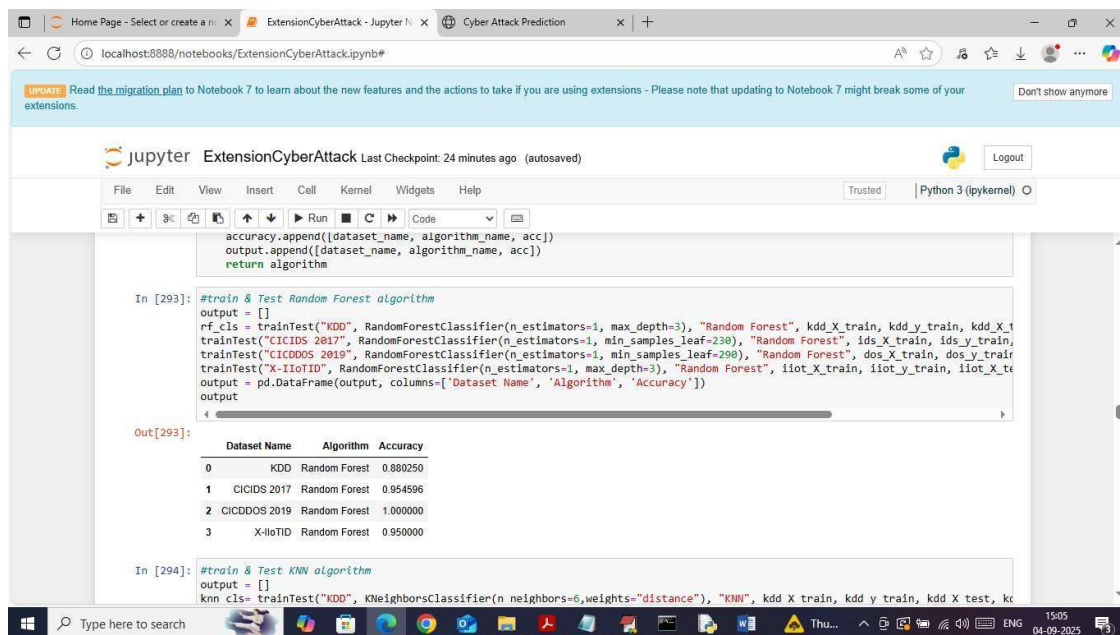
Out[284]:

	Date	Timestamp	Src_IP	Src_port	Des_IP	Des_port	Protocol	Service	Duration	Scr_bytes	OSSEC_alert_level	Login_attempt
0	9/01/2020	1578540956	192.168.2.199	49278	192.168.2.10	80	TCP	http	0.67369	13437	5	0
1	13/01/2020	1578871873	10.0.1.5	39769	131.236.3.92	53	UDP	dns	0.000083	78	0	0
2	9/01/2020	1578522486	172.24.1.80	59050	172.24.1.1	53	UDP	dns	0.000132	38	0	0
3	27/02/2020	1582757640	192.168.2.196	37966	192.168.2.10	1880	TCP	websocket	9.378481	1121	0	1
4	16/12/2019	1576452612	172.24.1.80	38233	172.24.1.1	53	UDP	dns	0.000074	+	0	0
9995	16/12/2019	1576629009	192.168.10.155	41254	192.168.2.10	80	TCP	http	7.515996	+	0	0
9996	24/02/2020	1582504310	172.24.1.244	60693	172.24.1.1	5683	UDP	coap	-	+	0	0
9997	24/02/2020	1582504507	172.24.1.233	53426	172.24.1.1	5683	UDP	coap	67.4439	1686	0	0
9998	16/12/2019	1576632817	10.0.1.5	52526	129.94.0.197	53	UDP	dns	0.000032	74	0	0
9999	7/01/2020	1578375277	192.168.2.199	57602	192.168.2.10	1152	TCP	other	0.000018	+	0	0

Screen 4 Displaying XIOTIOT dataset values



Screen 5 Visualizing graph of different attacks found in dataset



Screen 6 Training Random Forest

Home Page - Select or create a notebook | ExtensionCyberAttack - Jupyter Notebook | Cyber Attack Prediction

localhost:8888/notebooks/ExtensionCyberAttack.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 25 minutes ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

2 CICDDOS 2019 Random Forest 1.000000
3 X-IIoTID Random Forest 0.950000

In [294]: #train & Test KNN algorithm
output = []
knn_cls = trainTest("KDD", KNeighborsClassifier(n_neighbors=6, weights="distance"), "KNN", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", KNeighborsClassifier(n_neighbors=60, leaf_size=120), "KNN", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", KNeighborsClassifier(n_neighbors=90, leaf_size=320), "KNN", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", KNeighborsClassifier(n_neighbors=6, weights="distance"), "KNN", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output

Out[294]:

	Dataset Name	Algorithm	Accuracy
0	KDD	KNN	0.993250
1	CICIDS 2017	KNN	0.983389
2	CICDDOS 2019	KNN	0.984981
3	X-IIoTID	KNN	0.984000

In [295]: #train & Test SVM algorithm
output = []

Screen 7 KNN

Home Page - Select or create a notebook | ExtensionCyberAttack - Jupyter Notebook | Cyber Attack Prediction

localhost:8888/notebooks/ExtensionCyberAttack.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 26 minutes ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

In [295]: #train & Test SVM algorithm
output = []
trainTest("KDD", svm.SVC(kernel="linear"), "SVM", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", svm.SVC(kernel="linear", tol=2.001), "SVM", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", svm.SVC(kernel="linear", tol=2.001), "SVM", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", svm.SVC(kernel="linear"), "SVM", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output

Out[295]:

	Dataset Name	Algorithm	Accuracy
0	KDD	SVM	0.988250
1	CICIDS 2017	SVM	0.429679
2	CICDDOS 2019	SVM	0.458073
3	X-IIoTID	SVM	0.973500

In [296]: #train & Test MLP algorithm
output = []
mlp_cls = trainTest("KDD", MLPClassifier(tol=4), "MLP", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", MLPClassifier(tol=3, solver="lbfgs"), "MLP", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", MLPClassifier(tol=3, solver="lbfgs"), "MLP", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", MLPClassifier(tol=18, solver="sgd"), "MLP", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output

Screen 8 SVM

Home Page - Select or create a notebook | ExtensionCyberAttack - Jupyter | Cyber Attack Prediction

localhost:8888/notebooks/ExtensionCyberAttack.ipynb*

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 26 minutes ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

In [296]:

```
#train & Test MLP algorithm
output = []
mlp_cls = trainTest("KDD",MLPClassifier(tol=4), "MLP", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", MLPClassifier(tol=3,solver="lbfgs"), "MLP", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", MLPClassifier(tol=3,solver="lbfgs"), "MLP", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", MLPClassifier(tol=10,solver="sgd"), "MLP", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[296]:

	Dataset Name	Algorithm	Accuracy
0	KDD	MLP	0.986250
1	CICIDS 2017	MLP	0.521595
2	CICDDOS 2019	MLP	0.680851
3	X-IIoTID	MLP	0.932000

In [297]:

```
#train & Test LogisticRegression algorithm
output = []
trainTest("KDD",LogisticRegression(solver="liblinear"), "Logistic Regression", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", LogisticRegression(solver="liblinear",tol=3), "Logistic Regression", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", LogisticRegression(solver="liblinear",tol=5), "Logistic Regression", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", LogisticRegression(solver="liblinear"), "Logistic Regression", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Screen 9 MPL

Home Page - Select or create a notebook | ExtensionCyberAttack - Jupyter | Cyber Attack Prediction

localhost:8888/notebooks/ExtensionCyberAttack.ipynb*

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 27 minutes ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

In [297]:

```
#train & Test LogisticRegression algorithm
output = []
trainTest("KDD",LogisticRegression(solver="liblinear"), "Logistic Regression", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", LogisticRegression(solver="liblinear",tol=3), "Logistic Regression", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", LogisticRegression(solver="liblinear",tol=5), "Logistic Regression", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", LogisticRegression(solver="liblinear"), "Logistic Regression", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[297]:

	Dataset Name	Algorithm	Accuracy
0	KDD	Logistic Regression	0.964750
1	CICIDS 2017	Logistic Regression	0.570321
2	CICDDOS 2019	Logistic Regression	0.541927
3	X-IIoTID	Logistic Regression	0.960000

In [298]:

```
#train & Test XGBoost algorithm
output = []
trainTest("KDD", GaussianNB(var_smoothing=1e-09), "Naive Bayes", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", GaussianNB(var_smoothing=1e-03), "Naive Bayes", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", GaussianNB(var_smoothing=1e-01), "Naive Bayes", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", GaussianNB(var_smoothing=1e-03), "Naive Bayes", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Screen 10 Logistic Regression

Home Page - Select or create a notebook | ExtensionCyberAttack - Jupyter Notebook | Cyber Attack Prediction

localhost:8888/notebooks/ExtensionCyberAttack.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 27 minutes ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (pykernel)

1 CICIDS 2017 Logistic Regression 0.570321
2 CICDDOS 2019 Logistic Regression 0.541927
3 X-IIoTID Logistic Regression 0.960000

In [298]:

```
#train & Test XGBoost algorithm
output = []
trainTest("KDD", GaussianNB(var_smoothing=1e-09), "Naive Bayes", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", GaussianNB(var_smoothing=1e-03), "Naive Bayes", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", GaussianNB(var_smoothing=1e-01), "Naive Bayes", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", GaussianNB(var_smoothing=1e-03), "Naive Bayes", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[298]:

	Dataset Name	Algorithm	Accuracy
0	KDD	Naive Bayes	0.633250
1	CICIDS 2017	Naive Bayes	0.959025
2	CICDDOS 2019	Naive Bayes	0.961202
3	X-IIoTID	Naive Bayes	0.981500

In [299]:

```
def trainDNN(dataset_name, algorithm_name, X_train, y_train, X_test, y_test, output, model_name):
```

Screen 11 Naïve Bayes

Home Page - Select or create a notebook | ExtensionCyberAttack - Jupyter Notebook | Cyber Attack Prediction

localhost:8888/notebooks/ExtensionCyberAttack.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 28 minutes ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (pykernel)

In [300]:

```
#train & Test XGBoost algorithm
output = []
trainDNN("KDD", "DNN", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output, "kdd_weight.hdfs")
trainDNN("CICIDS 2017", "DNN", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output, "ids_weight.hdfs")
trainDNN("CICDDOS 2019", "DNN", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output, "dos_weight.hdfs")
trainDNN("X-IIoTID", "DNN", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output, "iot_weight.hdfs")
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[300]:

	Dataset Name	Algorithm	Accuracy
0	KDD	DNN	0.987938
1	CICIDS 2017	DNN	0.967885
2	CICDDOS 2019	DNN	0.967459
3	X-IIoTID	DNN	0.977500

In [301]:

```
#training stacking extension hybrid model by combining multiple algorithms such as MLP, Random Forest and KNN
rf = RandomForestClassifier()
mlp = MLPClassifier()
knn = KNeighborsClassifier()
estimators = [('rf', rf), ('mlp', mlp), ('knn', knn)]
```

Screen 12 DNN

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 29 minutes ago (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (pykernel)

```
In [301]: #training stacking extension hybrid model by combining multiple algorithms such as MLP, Random Forest and KNN
rf = RandomForestClassifier()#defining base model
mlp = MLPClassifier()
knn = KNeighborsClassifier()
estimators = [('rf', rf), ('mlp', mlp), ('knn', knn)]
hybrid_model = StackingClassifier(estimators=estimators, final_estimator=rf)#creating extension stack model with based algorithm
output = []
ext_model = trainTest("KDD", hybrid_model, "Hybrid Stack Model", kdd_X_train, kdd_y_train, kdd_X_train, kdd_y_train, output)
hybrid_model = StackingClassifier(estimators=estimators, final_estimator=rf)
trainTest("CICIDS 2017", hybrid_model, "Hybrid Stack Model", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", hybrid_model, "Hybrid Stack Model", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IloTID", hybrid_model, "Hybrid Stack Model", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[301]:

	Dataset Name	Algorithm	Accuracy
0	KDD	Hybrid Stack Model	0.999062
1	CICIDS 2017	Hybrid Stack Model	1.000000
2	CICDDOS 2019	Hybrid Stack Model	1.000000
3	X-IloTID	Hybrid Stack Model	1.000000

Screen 13 Training extension hybrid algorithms

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter ExtensionCyberAttack Last Checkpoint: 30 minutes ago (autosaved)

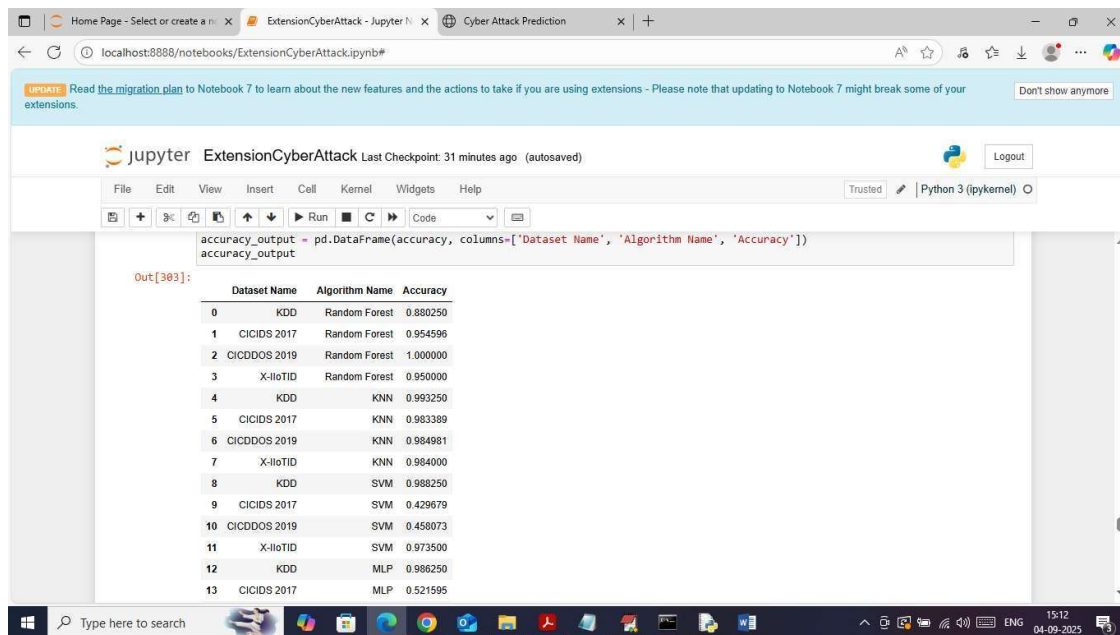
File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (pykernel)

```
#correct prediction
shap.initjs()
explainer = shap.TreeExplainer(rf_cls, kdd_X_train)
# Explain the predictions of your model
shap_values = explainer.shap_values(kdd_X_test, check_additivity=False)
#summary plot to explain names of features which is contributing most for the algorithm to make correct prediction
shap.summary_plot(shap_values[0], kdd_X_test, feature_names=columns1)
```

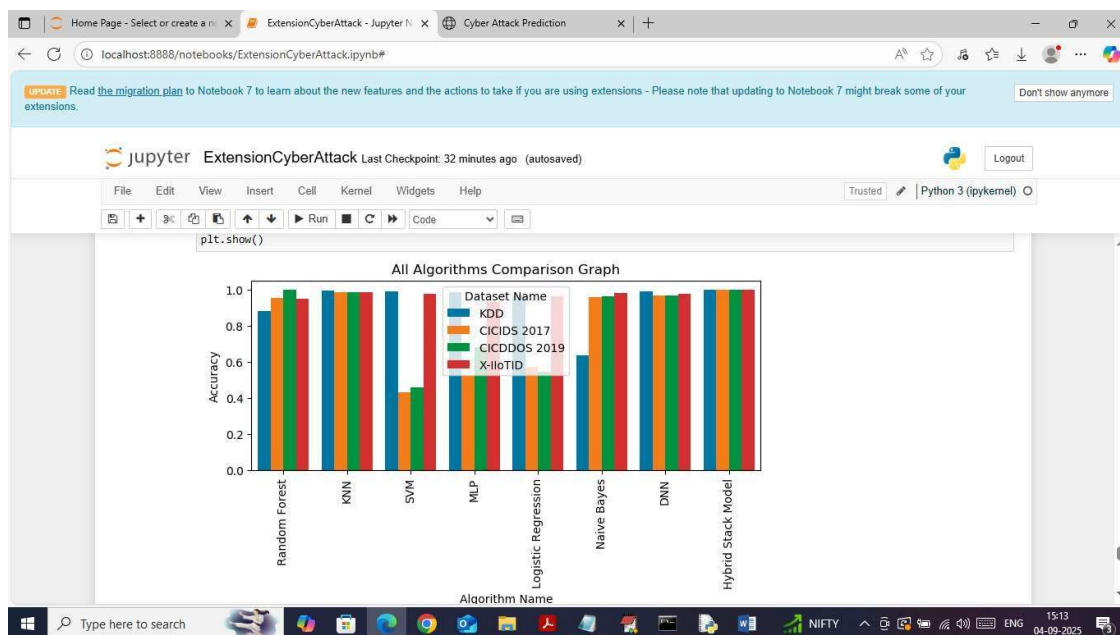
dst_bytes
rerror_rate
num_compromised
srv_count
dst_host_srv_error_rate
num_failed_logins
num_file_creations

High

Screen 14 SHAP explanation tool



Screen 15 Showing all algorithms accuracy



Screen 16 Showing comparison graph between all algorithms

```
In [305]: #perform prediction on test data
testData = pd.read_csv("Dataset/testData.csv")
data = testData.copy().values
for i in range(len(label_encoder1)-1):#Label encoding from non-numeric to numeric
    le = label_encoder1[i]
    testData[le[0]] = pd.Series(le[1].transform(testData[le[0]].astype(str)))#encode all str columns to numeric
testData.fillna(0, inplace = True)#replace missing values with 0
testData = testData.values
testData = scaler1.transform(testData) #normalize features
predict = ext_model.predict(testData)
for i in range(len(predict)):
    print("Test Data = "+str(data[i])+" Predicted Cyber Attack ==> "+kdd_labels[predict[i]])
    print()

Test Data = [0 'tcp' 'http' 'SF' 264 3011 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0
0 0 0 0 1 0 0 0 0 0 255 241 0 95 0 02 0 0 0 0 0 0 0 0 0 0 0 0] Predicted Cyber Attack ==> normal

Test Data = [0 'tcp' 'other' 'SF' 3775 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 20 1 0 05 0 0
0 0 0 0 0 05 0 75 0 0 255 1 0 0 0 07 0 45 0 0 0 0 0 0 0 0 0 0] Predicted Cyber Attack ==> normal

Test Data = [0 'tcp' 'private' 'RE3' 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 238 1 0 11
0 0 0 09 1 0 0 0 1 0 0 0 255 1 0 0 0 97 0 0 0 0 0 1 0 0 0 04 1 0] Predicted Cyber Attack ==> Satan

Test Data = [0 'tcp' 'pop_2' 'S0' 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 241 9 1 0 1 0
0 0 0 0 0 04 0 06 0 0 255 7 0 03 0 08 0 0 0 0 1 0 1 0 0 0 0 0] Predicted Cyber Attack ==> neptune
```

Screen 17 Attack prediction

15. CONCLUSION

This project successfully demonstrates the effectiveness of integrating multiple machine learning and deep learning algorithms through a hybrid stacked model for cyber-attack prediction. The results show that combining models like MLP, KNN, and Random Forest significantly enhances accuracy and reliability compared to individual algorithms. By training on diverse benchmark datasets such as NSL-KDD, CICIDS2017, CICDDOS2019, and XIIOT, the system achieves high adaptability and generalization to various attack types. The inclusion of SHAP explainability tools ensures transparency, allowing users to understand which features influence predictions most. The model's web-based interface further makes real-time attack detection practical and user-friendly. Overall, the proposed approach presents a robust, accurate, and interpretable framework for modern cybersecurity systems. It bridges the gap between high performance and model transparency, offering a reliable foundation for future intelligent threat detection and defense mechanisms in dynamic digital environments.

FUTURE SCOPE

In the future, this work can be extended by integrating advanced deep learning models like transformers and graph neural networks to detect complex attack patterns. Incorporating real-time streaming data and cloud-based monitoring will enhance scalability. The hybrid model can also be improved using adversarial learning and multimodal analysis, enabling faster and more adaptive responses to evolving cyber threats in dynamic network environments.

16. BIBLIOGRAPHY

- [1] M. Alazab, R. Latif, M. Alazab, S. R. Islam, and S. Venkatraman, "Towards cybersecurity prediction systems using machine learning: An overview," *IEEE Access*, vol. 11, pp. 11345–11367, 2023.
- [2] S. Yadav, R. Patel, and S. Patel, "Comprehensive review on cyberattack prediction using AI," *Comput. Electr. Eng.*, vol. 112, Art. no. 108129, 2023.
- [3] A. S. Kumbhare, P. A. Kharkar, and S. R. Sakhare, "Cyberattack detection and prediction using deep learning models," *J. Inf. Secur. Appl.*, vol. 79, Art. no. 103468, 2023.
- [4] R. Alkhadra, J. Abuzaid, M. AlShammari, and N. Mohammad, "Solarwinds hack: In-depth analysis and countermeasures," in *Proc. 12th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Jul. 2021, pp. 1–7.
- [5] Cobalt. (Jun. 20, 2024). 11 Biggest Cybersecurity Attacks in History. [Online]. Available: <https://www.cobalt.io/blog/biggest-cybersecurity-attacks-in-history>
- [6] D.-Y. Kao, S.-C. Hsiao, and R. Tso, "Analyzing WannaCry ransomware considering the weapons and exploits," in *Proc. 21st Int. Conf. Adv. Commun. Technol. (ICACT)*, Feb. 2019, pp. 1098–1107.
- [7] K. Bresniker, A. Gavrilovska, J. Holt, D. Milojicic, and T. Tran, "Grand challenge: Applying artificial intelligence and machine learning to cybersecurity," *Computer*, vol. 52, no. 12, pp. 45–52, Dec. 2019.
- [8] M. Husák, J. Komárková, E. Bou-Harb, and P. Celeda, "Survey of attack projection, prediction, and forecasting in cyber security," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 640–660, 1st Quart., 2019.
- [9] N. Mohamed, "Current trends in AI and ML for cybersecurity: A state-of-the-art survey," *Cogent Eng.*, vol. 10, no. 2, 2023.
- [10] L. Chan et al., "Survey of AI in cybersecurity for information technology management," in *Proc. IEEE TEMSCON*, Jun. 2019, pp. 1–8.
- [11] G. Disterer, "ISO/IEC 27000, 27001 and 27002 for information security management," *J. Inf. Secur.*, vol. 4, no. 2, pp. 92–100, 2013.

- [12] J. H. Li, "Cyber security meets artificial intelligence: A survey," *Frontiers Inf. Technol. Electron. Eng.*, vol. 19, no. 12, pp. 1462–1474, 2018.
- [13] I. H. Sarker et al., "Cybersecurity data science: An overview from machine learning perspective," *J. Big Data*, vol. 7, pp. 1–29, May 2020.
- [14] M. K. Hassan et al., "An improved hybrid deep learning model for cyber-attack detection," *IEEE Access*, vol. 10, pp. 54456–54470, 2022.
- [15] S. Ankalaki, S. J. Raj, and P. K. Karthik, "A hybrid deep ensemble learning approach for predicting DDoS attacks," *Comput. Netw.*, vol. 229, Art. no. 109732, 2023.
- [16] T. M. Mitchell, *Machine Learning*, New York, NY, USA: McGraw-Hill, 1997.
- [17] R. E. Schapire, "The boosting approach to machine learning: An overview," *Nonlinear Estimation Classification*, vol. 171, pp. 149–171, 2003.
- [18] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, 2001.
- [19] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.
- [20] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Netw.*, vol. 61, pp. 85–117, Jan. 2015.
- [21] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [22] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [23] M. A. Ferrag, L. Maglaras, S. Moschogiannis, and H. Janicke, "Deep learning for cyber security intrusion detection," *J. Inf. Secur. Appl.*, vol. 50, Feb. 2020.
- [24] S. K. Pandey et al., "Hybrid ML models for cyber attack prediction using CICIDS datasets," *IEEE Access*, vol. 11, pp. 34713–34726, 2023.
- [25] C. Yin, Y. Zhu, J. Fei, and X. He, "A deep learning approach for intrusion detection using recurrent neural networks," *IEEE Access*, vol. 5, pp. 21954–21961, 2017.

- [26] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 dataset," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Jul. 2009, pp. 1–6.
- [27] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset (CICIDS2017)," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
- [28] M. Ring et al., "A survey of network-based intrusion detection datasets," *Comput. Secur.*, vol. 86, pp. 147–167, Sep. 2019.
- [29] Y. Meidan et al., "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Jul. 2018.
- [30] B. Hitaj, P. Gasti, G. Ateniese, and F. Perez-Cruz, "PassGAN: A deep learning approach for password guessing," *arXiv:1709.00440*, 2017.
- [31] P. Dhoni and R. Kumar, "Synergizing generative AI and cybersecurity," *Authorea Preprints*, Aug. 2023.
- [32] J. J. Plotnek and J. Slay, "Cyber terrorism: A homogenized taxonomy and definition," *Comput. Secur.*, vol. 102, Mar. 2021.
- [33] S. Kim et al., "Cyber attack taxonomy for digital environment in nuclear power plants," *Nucl. Eng. Technol.*, vol. 52, no. 5, pp. 995–1001, May 2020.
- [34] M. Wu and Y. B. Moon, "Taxonomy of cross-domain attacks on cyber-manufacturing system," *Proc. Comput. Sci.*, vol. 114, pp. 367–374, 2017.
- [35] R. Heartfield et al., "A taxonomy of cyber-physical threats and impact in the smart home," *Comput. Secur.*, vol. 78, pp. 398–428, Sep. 2018.
- [36] Z. Alkhalil et al., "Phishing attacks: A recent comprehensive study and a new anatomy," *Frontiers Comput. Sci.*, vol. 3, 2021.
- [37] N. Z. Gorment et al., "Machine learning algorithm for malware detection: Taxonomy, current challenges, and future directions," *IEEE Access*, vol. 11, pp. 141045–141089, 2023.
- [38] M. A. Ferrag et al., "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, 2020.

- [39] M. Ring et al., “A survey of network-based intrusion detection data sets,” *Comput. Secur.*, vol. 86, pp. 147–167, 2019.
- [40] MIT Lincoln Laboratory, 1998 DARPA Intrusion Detection Evaluation Dataset, 2020. [Online]. Available: <https://www.ll.mit.edu>
- [41] KDD Cup 1999 Data, Univ. California, Irvine, CA, USA, Oct. 1999. [Online]. Available: <http://kdd.ics.uci.edu>
- [42] A. Sperotto et al., “A labeled data set for flow-based intrusion detection,” in *Proc. Int. Workshop IP Oper. Manag.*, Springer, 2009, pp. 39–50.
- [43] B. Sangster et al., “Toward instrumenting network warfare competitions to generate labeled datasets,” in *Proc. 2nd Conf. Cyber Secur. Experimentation Test*, 2009, pp. 1–15.
- [44] M. Tavallaei et al., “Toward developing a systematic approach to generate benchmark datasets for intrusion detection,” *Comput. Secur.*, vol. 31, no. 3, pp. 357–374, 2012.
- [45] S. Saad et al., “Detecting P2P botnets through network behavior analysis and machine learning,” in *Proc. 9th Annu. Int. Conf. Privacy, Secur. Trust*, Jul. 2011, pp. 174–180.
- [46] S. Bhattacharya and S. Selvakumar, “SSENet-2014 dataset: A dataset for detection of multiconnection attacks,” in *Proc. 3rd Int. Conf. Eco-Friendly Comput. Commun. Syst.*, Dec. 2014.
- [47] H. H. Jazi et al., “Detecting HTTP-based application layer DoS attacks on web servers,” *Comput. Netw.*, vol. 121, pp. 25–36, Jul. 2017.
- [48] A. Shiravi et al., “Toward developing a systematic approach to generate benchmark datasets for intrusion detection,” *Comput. Secur.*, vol. 31, no. 3, pp. 357–374, May 2012.
- [49] M. Bhuyan et al., “Towards generating real-life datasets for network intrusion detection,” *Int. J. Netw. Secur.*, vol. 17, pp. 683–701, Jan. 2015.
- [50] J. J. Santanna et al., “Booters—An analysis of DDoS-as-a-service attacks,” in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manage. (IM)*, May 2015, pp. 243–251.